

Two Pointer Templates

Two patterns — pick based on whether pointers converge or march together.

Quick Reference Table

#	Name	Description	Intuition	Time	Space	Key Implementation
167	Two Sum II	Sorted array. Find two numbers summing to target. Return 1-based indices.	Squeeze from both ends; each comparison tells you which side is wrong, so one move eliminates a whole row/column of pairs.	$O(n)$	$O(1)$	Standard
15	3Sum	Find all unique triplets summing to 0. No duplicate triplets in output.	Pin one number, then it becomes Two Sum on the rest — turning a triple search into a sweep.	$O(n^2)$	$O(1)$ excluding output ($O(n)$ sort stack)	Standard
18	4Sum	Find all unique quadruplets summing to target.	Pin two numbers, then the inner search is Two Sum — same idea as 3Sum with one more fixed level.	$O(n^3)$	$O(1)$ excluding output ($O(n)$ sort stack)	Standard
11	Container With Most Water	Given heights of vertical lines, find two lines forming the container with most water.	Width shrinks every step, so only a taller boundary can ever help; abandon the shorter wall.	$O(n)$	$O(1)$	Standard
42	Trapping Rain Water	Given elevation heights, compute total water trapped after rain.	Water level at a cell is set by the shorter side's tallest wall, so always process from the lower side where that max is already known.	$O(n)$	$O(1)$	Standard
27	Remove Element	Remove all occurrences of <code>val</code> in-place. Return new length.	The writer only advances on keepers, so it always points at the next free slot for a kept value.	$O(n)$	$O(1)$	Standard
26	Remove Duplicates from Sorted Array	Remove duplicates in-place so each element appears at most once. Return new length.	Since the array is sorted, comparing against the last written value is enough to drop every duplicate run.	$O(n)$	$O(1)$	Standard
80	Remove Duplicates from Sorted Array II	Each element may appear at most twice in-place. Return new length.	Looking two slots back lets at most two copies through before the third is rejected.	$O(n)$	$O(1)$	Standard
283	Move Zeroes	Move all zeroes to end while preserving relative order of non-zero elements.	Compact the non-zeros to the front in order, then pad the leftover tail with zeros.	$O(n)$	$O(1)$	Standard
75	Sort Colors	Sort array of 0s, 1s, 2s in-place without library sort.	A single scan grows a "small" region from the left and a "large" region from the right, leaving mids in the middle.	$O(n)$	$O(1)$	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
1	Two Sum	Given an integer array and a target sum, return the indices of two numbers that add up to target. Exactly one solution exists.	Trade space for time — remember every value you've passed so the complement is a single lookup.	$O(n)$	$O(n)$	Standard
16	3Sum Closest	Find three integers whose sum is closest to target. Return that sum.	Same pin-and-sweep as 3Sum, but instead of matching exactly you keep the running closest.	$O(n^2)$	$O(1)$	Standard
31	Next Permutation	Rearrange the array to its lexicographically next greater permutation in-place. If none, sort ascending.	The longest decreasing suffix is already maximal; bump the digit just before it up to the smallest larger value, then reset the suffix to ascending.	$O(n)$	$O(1)$	Standard
41	First Missing Positive	Find the smallest missing positive integer in an unsorted array. $O(n)$ time, $O(1)$ space.	Use the array itself as a hash table keyed by value, then scan for the first slot holding the wrong number.	$O(n)$	$O(1)$	Standard
48	Rotate Image	Rotate an $n \times n$ matrix 90 degrees clockwise in-place.	Transposing flips across the main diagonal; reversing each row then completes the clockwise turn.	$O(n^2)$	$O(1)$	Standard
54	Spiral Matrix	Return all elements of an $m \times n$ matrix in spiral order.	Peel the matrix like an onion, one boundary layer per loop.	$O(m \cdot n)$	$O(1)$ excluding output	Standard
73	Set Matrix Zeroes	If any element is 0, set its entire row and column to 0, in-place.	Reuse the first row and column as the bookkeeping space so no extra arrays are needed.	$O(m \cdot n)$	$O(1)$	Standard
128	Longest Consecutive Sequence	Find the length of the longest run of consecutive integers in an unsorted array. $O(n)$.	Each sequence is counted exactly once by starting only at its smallest member.	$O(n)$	$O(n)$	Standard
164	Maximum Gap	Find the maximum gap between successive elements in the sorted form of an array. $O(n)$.	With buckets sized at the minimum possible gap, the answer never lies inside a bucket, only between bucket boundaries.	$O(n)$	$O(n)$	Standard
169	Majority Element	Find the element appearing more than $n/2$ times.	Pairing off different elements cancels them out; the majority element always survives.	$O(n)$	$O(1)$	Standard
189	Rotate Array	Rotate the array right by k positions, in-place.	Two partial reversals after a full reversal place each block in its	$O(n)$	$O(1)$	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
			rotated position without extra space.			
217	Contains Duplicate	Return true if any value appears at least twice.	A set rejects repeats, so the first failed insert proves a duplicate.	$O(n)$	$O(n)$	Standard
220	Contains Duplicate III	Are there indices i, j with $ \text{nums}[i] - \text{nums}[j] \leq \text{valueDiff}$ and $ i - j \leq \text{indexDiff}$?	Same-bucket means automatically within range; neighbor buckets need an explicit check. Slide a window of size indexDiff .	$O(n)$	$O(\min(n, \text{indexDiff}))$	Standard
229	Majority Element II	Find all elements appearing more than $n/3$ times.	At most two elements can each occur more than a third of the time, so track two running candidates.	$O(n)$	$O(1)$	Standard
238	Product of Array Except Self	Return output where $\text{output}[i]$ is the product of all elements except $\text{nums}[i]$. No division.	Each position's answer is everything to its left times everything to its right.	$O(n)$	$O(1)$ excluding output	Standard
259	3Sum Smaller	Count triplets with sum strictly less than target in a sorted array.	Once the largest partner works, every smaller partner does too, so count them in bulk.	$O(n^2)$	$O(1)$	bulk-count all valid j for this i
280	Wiggle Sort	Reorder in-place so $\text{nums}[0] \leq \text{nums}[1] \geq \text{nums}[2] \leq \text{nums}[3] \dots$	A greedy local swap is always safe — it never breaks the pair you already fixed.	$O(n)$	$O(1)$	Standard
289	Game of Life	Simulate one Game of Life step, updating all cells simultaneously, in-place.	Pack the new state into a spare bit so neighbor counts still read the old state during the pass.	$O(m \cdot n)$	$O(1)$	Standard
303	Range Sum Query - Immutable	Answer many range-sum queries on a static array efficiently.	A range sum is the difference of two prefix sums, so each query is $O(1)$ after one build pass.	$O(n)$ build, $O(1)$ query	$O(n)$	Standard
304	Range Sum Query 2D - Immutable	Answer many 2D region-sum queries on a static matrix efficiently.	Each region sum combines four corner prefix sums via inclusion-exclusion.	$O(m \cdot n)$ build, $O(1)$ query	$O(m \cdot n)$	Standard
307	Range Sum Query - Mutable	Support range-sum queries and point updates.	A Fenwick tree stores partial sums over power-of-two ranges so updates and queries each touch only $\log n$ nodes.	$O(\log n)$ update/query	$O(n)$	Standard
311	Sparse Matrix Multiplication	Multiply two sparse matrices, skipping zero elements.	A zero in A contributes nothing, so skip it entirely instead of doing a full triple loop.	$O(mkn)$ worst case	$O(m \cdot n)$ excluding output	skip zero to exploit sparsity
315	Count of Smaller Numbers After	For each element, count elements to its right that are smaller. $O(n \log n)$	During a merge, every time a right element is	$O(n \log n)$	$O(n)$	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
	Self	n).	taken before a left one, it is a smaller-to-the-right for that left element.			
325	Maximum Size Subarray Sum Equals k	Find the longest subarray with sum equal to k. $O(n)$.	A subarray sums to k exactly when two prefix sums differ by k; keep the earliest index to maximize length.	$O(n)$	$O(n)$	Standard
327	Count of Range Sum	Count range sums that lie in [lower, upper]. $O(n \log n)$.	A range sum is a difference of two prefix sums; merge sort lets you count qualifying pairs across halves efficiently.	$O(n \log n)$	$O(n)$	Standard
334	Increasing Triplet Subsequence	Return true if there exists $i < j < k$ with <code>nums[i] < nums[j] < nums[k]</code> .	Maintaining the two smallest ascending values seen so far is enough to detect a third larger one.	$O(n)$	$O(1)$	Standard
349	Intersection of Two Arrays	Return the intersection of two arrays (unique elements only).	Set membership turns intersection into linear lookups.	$O(n + m)$	$O(n)$	Standard
350	Intersection of Two Arrays II	Return the intersection including duplicates (multiplicity is the min of counts).	A frequency map lets each shared occurrence be matched at most once.	$O(n + m)$	$O(\min(n, m))$	Standard
370	Range Addition	Apply range updates <code>[start, end] += inc</code> then return the final array.	Mark only the endpoints of each update; a single prefix pass materializes all increments.	$O(n + \text{updates})$	$O(n)$	Standard
384	Shuffle an Array	Return a uniformly random permutation of the array; support reset to original.	Choosing each position's element uniformly from the remaining ones yields every permutation with equal probability.	$O(n)$ per shuffle	$O(n)$	Standard
414	Third Maximum Number	Return the third distinct maximum; if it doesn't exist, return the maximum.	Keep a sorted top-three as you scan, skipping duplicates.	$O(n)$	$O(1)$	Standard
419	Battleships in a Board	Count battleships ('X' runs, horizontal or vertical, never adjacent).	Each ship has exactly one cell with no ship-neighbor above or to its left, so count those.	$O(m \cdot n)$	$O(1)$	Standard
442	Find All Duplicates in an Array	Values in $[1, n]$, each appears once or twice. Return the ones appearing twice. $O(n)$ time, $O(1)$ extra space.	Flip the sign at each value's home index; encountering an already-negative slot means that value repeated.	$O(n)$	$O(1)$	Standard
448	Find All Numbers Disappeared in an Array	Values in $[1, n]$. Return all numbers in $[1, n]$ missing from the array.	Use sign marking; any home index never marked means that number was absent.	$O(n)$	$O(1)$	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
454	4Sum II	Count quadruplets (i,j,k,l) with <code>A[i]+B[j]+C[k]+D[l] == 0</code> .	Split four arrays into two pairs so a hash of one pair's sums turns the search into $O(n^2)$ lookups.	$O(n^2)$	$O(n^2)$	Standard
457	Circular Array Loop	Detect a cycle of length > 1 with consistent direction in a circular array of jumps.	Fast/slow pointers detect cycles; reject single-element self-loops and direction flips.	$O(n)$	$O(1)$	Standard
485	Max Consecutive Ones	Find the maximum number of consecutive 1s in a binary array.	Grow a streak while seeing 1s and reset at each 0.	$O(n)$	$O(1)$	Standard
493	Reverse Pairs	Count pairs (i,j) with <code>i < j</code> and <code>nums[i] > 2 * nums[j]</code> .	Sorted halves let you count <code>nums[i] > 2*nums[j]</code> pairs with a linear two-pointer sweep per merge.	$O(n \log n)$	$O(n)$	Standard
498	Diagonal Traverse	Traverse an $m \times n$ matrix diagonally, alternating direction each diagonal.	Cells on the same diagonal share <code>r+c</code> ; flip the read direction on alternating diagonals to zig-zag.	$O(m*n)$	$O(1)$ excluding output	Standard
517	Super Washing Machines	Minimize the max number of moves to equalize dresses across machines (one dress to a neighbor per move).	A bottleneck is either a machine with too much surplus or a cut that must pass a large net flow.	$O(n)$	$O(1)$	Standard
523	Continuous Subarray Sum	Is there a subarray of length ≥ 2 whose sum is a multiple of k ?	Two prefix sums with the same remainder mod k bound a subarray sum divisible by k .	$O(n)$	$O(\min(n, k))$	Standard
525	Contiguous Array	Find the longest subarray with equal numbers of 0s and 1s.	Equal counts means a running balance returns to a value seen before.	$O(n)$	$O(n)$	Standard
532	K-diff Pairs in an Array	Count unique pairs (i,j) where <code>nums[i] - nums[j] == k</code> ($k \geq 0$).	Counting distinct values avoids duplicate pairs; the zero-diff case needs a repeated value.	$O(n)$	$O(n)$	Standard
560	Subarray Sum Equals K	Count subarrays summing to exactly k .	Each earlier prefix equal to <code>s-k</code> marks the start of a qualifying subarray ending here.	$O(n)$	$O(n)$	Standard
566	Reshape the Matrix	Reshape an $m \times n$ matrix into $r \times c$ keeping row-major order; return original if sizes mismatch.	Both shapes share a linear index, so map between them with division and modulo.	$O(m*n)$	$O(r*c)$ excluding output	Standard
581	Shortest Unsorted Continuous Subarray	Find the shortest subarray that, if sorted, makes the whole array sorted.	The window starts where an element exceeds some later minimum and ends where an element falls	$O(n)$	$O(1)$	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
			below some earlier maximum.			
594	Longest Harmonious Subsequence	Find the longest subsequence where max and min differ by exactly 1.	A harmonious subsequence uses exactly two adjacent values, so combine each pair's counts.	$O(n)$	$O(n)$	Standard
599	Minimum Index Sum of Two Lists	Find common strings between two lists with minimum index sum.	Index sum is minimized greedily as you scan; ties are collected together.	$O(n + m)$	$O(n)$	Standard
611	Valid Triangle Number	Count triplets from an array of side lengths that form a valid triangle.	Only the two smaller sides need to beat the largest, so fix the largest and bulk-count valid pairs.	$O(n^2)$	$O(1)$	bulk-count valid i for this j
661	Image Smoother	Replace each cell with the floor of the average of itself and its up-to-8 neighbors.	Average over the in-bounds 3×3 window centered on each cell.	$O(m \cdot n)$	$O(m \cdot n)$ excluding output	Standard
723	Candy Crush	Repeatedly crush horizontal/vertical runs of 3+ same-positive candies and apply gravity, until stable.	Mark all matches in one pass before clearing so simultaneous crushes are handled, then let candies fall.	$O((m \cdot n)^2)$ worst case	$O(1)$	Standard
724	Find Pivot Index	Find the leftmost index where the sum to its left equals the sum to its right.	Track a running left sum and derive the right sum from the precomputed total.	$O(n)$	$O(1)$	Standard
766	Toeplitz Matrix	Check if every top-left to bottom-right diagonal has a single value.	Comparing each cell to its diagonal predecessor verifies all diagonals locally.	$O(m \cdot n)$	$O(1)$	Standard
769	Max Chunks To Make Sorted	Array is a permutation of $[0, n-1]$. Max number of chunks that can be sorted independently to sort the whole.	When the largest value seen so far equals the index, everything up to here is a self-contained block.	$O(n)$	$O(1)$	Standard
795	Number of Subarrays with Bounded Maximum	Count subarrays whose maximum lies in $[left, right]$.	"Max in range" equals the difference of two "max at most X" counts, each computed in one pass.	$O(n)$	$O(1)$	Standard
807	Max Increase to Keep City Skyline	Increase building heights as much as possible without changing skylines viewed from any of 4 sides.	A building is capped by the smaller of its row and column maxima to preserve both silhouettes.	$O(n^2)$	$O(n)$	Standard
825	Friends Of Appropriate Ages	Count friend requests; x friends y unless $age[y] \leq 0.5 \cdot age[x] + 7$, $age[y] > age[x]$, or $age[y] > 100$ && $age[x] < 100$.	Ages are bounded, so counting per age-bucket pair collapses the problem to constant-size work.	$O(n + 120^2)$	$O(1)$	Standard
838	Push Dominoes	Simulate falling dominoes given a string of 'L', 'R', '.'.	Each cell's outcome is the balance of forces	$O(n)$	$O(n)$	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
			from the nearest pushers on either side.			
845	Longest Mountain in Array	Find the length of the longest mountain (strictly up then strictly down).	Scan to a peak by rising, then descend; the span counts only when both directions occurred.	$O(n)$	$O(1)$	Standard
896	Monotonic Array	Return true if the array is entirely non-increasing or entirely non-decreasing.	A single pass noting direction changes detects non-monotonicity.	$O(n)$	$O(1)$	Standard
912	Sort an Array	Sort an integer array. Implement an $O(n \log n)$ sort.	Recursively split, sort halves, and merge — stable $O(n \log n)$ without library sort.	$O(n \log n)$	$O(n)$	Standard
974	Subarray Sums Divisible by K	Count subarrays whose sum is divisible by k.	Two prefixes with the same remainder mod k bound a divisible subarray; count pairs per remainder.	$O(n)$	$O(k)$	Standard
977	Squares of a Sorted Array	Return the squares of a sorted array in sorted order.	The biggest squares sit at the extremes, so fill the result from the largest slot inward.	$O(n)$	$O(1)$ excluding output	Standard
1013	Partition Array Into Three Parts With Equal Sum	Can the array be split into three contiguous parts with equal sums?	Greedily close off a part each time the running sum hits one-third; succeed if at least three parts form.	$O(n)$	$O(1)$	Standard
1074	Number of Submatrices That Sum to Target	Count submatrices summing to target.	Reduce 2D to 1D by fixing the top and bottom rows, then count target-sum subarrays.	$O(m^2 * n)$	$O(n)$	Standard
1109	Corporate Flight Bookings	Given bookings [first, last, seats], return total seats booked per flight.	Range increments become two endpoint marks resolved by one prefix pass.	$O(n + \text{bookings})$	$O(n)$	Standard
1213	Intersection of Three Sorted Arrays	Return the common elements of three sorted arrays.	Like merging — advance whichever pointer lags so all three meet at shared values.	$O(n)$	$O(1)$ excluding output	Standard
1275	Find Winner on a Tic Tac Toe Game	Given alternating A/B moves, determine the winner, "Draw", or "Pending".	Signed counts per line reveal a win the moment any line reaches +3 or -3.	$O(1)$	$O(1)$	Standard
1351	Count Negative Numbers in a Sorted Matrix	Count negatives in a matrix sorted descending by row and column.	From the bottom-left, sorted order lets each step rule out a whole row or column of negatives.	$O(m + n)$	$O(1)$	Standard
1424	Diagonal Traverse II	Traverse a jagged 2D list diagonally from bottom-left to top-right.	Cells on a diagonal share $r + c$; emitting rows in descending order gives the	$O(\text{total})$	$O(\text{total})$	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
			bottom-left-up direction.			
1460	Make Two Arrays Equal by Reversing Sub-arrays	Can target become arr after reversing any sub-arrays any number of times?	Any permutation is reachable through reversals, so only element counts matter.	$O(n)$	$O(1)$	Standard
1480	Running Sum of 1d Array	Return the running (prefix) sum of the array.	Each running sum is the prior running sum plus the current value.	$O(n)$	$O(1)$	Standard
1498	Number of Subsequences That Satisfy the Given Sum Condition	Count subsequences where $\min + \max \leq \text{target}$. Answer mod $1e9+7$.	After sorting, fixing the minimum lets every subset of the elements up to the max be chosen freely.	$O(n \log n)$	$O(n)$	Standard
1748	Sum of Unique Elements	Sum all elements that appear exactly once.	Count occurrences, then sum only the singletons.	$O(n)$	$O(1)$	Standard
1762	Buildings With an Ocean View	Return indices of buildings with nothing taller to their right (ocean to the right).	Sweeping from the ocean side, only record buildings that exceed every taller one already passed.	$O(n)$	$O(1)$ excluding output	Standard
1868	Product of Two Run-Length Encoded Arrays	Multiply two run-length-encoded arrays element-wise; return the RLE product.	March through both run lists together, consuming the shorter overlap and coalescing equal results.	$O(n + m)$	$O(1)$ excluding output	Standard
1877	Minimize Maximum Pair Sum in Array	Pair up elements to minimize the maximum pair sum.	Pairing extremes balances the sums, keeping the largest pair as small as possible.	$O(n \log n)$	$O(1)$	Standard
1893	Check if All the Integers in a Range Are Covered	Are all integers in $[\text{left}, \text{right}]$ covered by at least one given interval?	Mark interval starts and ends, then a prefix sweep shows which points are covered.	$O(n + 50)$	$O(1)$	Standard
1894	Find the Student that Will Replace the Chalk	Students consume chalk in order cyclically; find who runs out.	Whole cycles repeat, so only the remainder after a full round determines the failing student.	$O(n)$	$O(1)$	Standard
1920	Build Array from Permutation	Return <code>result[i] = nums[nums[i]]</code> .	Each output is a double lookup into the permutation.	$O(n)$	$O(n)$ excluding output	Standard
1929	Concatenation of Array	Return <code>nums</code> concatenated with itself.	Copy each element into position <code>i</code> and <code>i+n</code> .	$O(n)$	$O(1)$ excluding output	Standard

Pattern 1 — Opposite Two Pointers

Variables: `i` = left pointer · `j` = right pointer · scan range `[i, j]` closed **Pseudocode:**

```
set i to the left end, j to the right end
while the pointers haven't crossed:
  if the current pair hits the target:
    record it, then step both ends inward
  else if the value is too small:
    move the left end right to grow it
  else (too large):
    move the right end left to shrink it
```

```
// MENTAL MODEL: two ends close in on the answer; each comparison rules out one whole end of the range.
// WHY sorted: moving inward only safely eliminates half the space if the array is sorted
int i = 0, j = n - 1;
while (i < j) {
  if (condition == target) {
    /* record */ i++; j--;
  } else if (tooSmall) {
    i++;
  } else {
    j--;
  }
}
```

Sort first. `i` and `j` converge inward. Used when the array is sorted and you can eliminate half the search space each step.

#167 Two Sum II

Description: Sorted array. Find two numbers summing to target. Return 1-based indices.

Key: sum too small → move `i` right. sum too large → move `j` left.

Intuition: Squeeze from both ends; each comparison tells you which side is wrong, so one move eliminates a whole row/column of pairs.

Variables: `i` = left pointer (1-based output `i+1`) · `j` = right pointer · search range `[i, j]` closed **Pseudocode:**

```
start i at the front, j at the back
while pointers haven't met:
  add the two ends
  if the sum equals target:
    return their 1-based indices
  else if the sum is too small:
    move left end right to increase the sum
  else:
    move right end left to decrease the sum
if nothing found, return sentinel
```

```

class Solution {
    public int[] twoSum(int[] nums, int target) {
        int i = 0, j = nums.length - 1;
        while (i < j) {
            int sum = nums[i] + nums[j];
            if (sum == target) {
                return new int[]{i + 1, j + 1};
            } else if (sum < target) {
                i++;
            } else {
                j--;
            }
        }
        return new int[]{-1, -1};
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#15 3Sum

Description: Find all unique triplets summing to 0. No duplicate triplets in output.

Key: sort first, fix `a`, run opposite two pointers on `[a+1, n-1]`. Skip duplicates at all three levels.

Intuition: Pin one number, then it becomes Two Sum on the rest — turning a triple search into a sweep.

Variables: `a` = fixed first index · `i / j` = inner left/right pointers over `[a+1, n-1]` closed · `sum` = triplet sum

Pseudocode:

```

sort so duplicates group and pointers can converge
prepare the result list
for each candidate first element a:
    skip a if it repeats the previous first element
    set inner pointers just after a and at the end
    while inner pointers haven't crossed:
        compute the triplet sum
        if it is zero:
            record the triplet and step both inner ends in

            skip duplicate left values
            skip duplicate right values
        else if too small:
            move left in
        else:
            move right in
return all triplets

```

```

class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> result = new ArrayList<>();
        for (int a = 0; a < nums.length - 2; a++) {
            if (a > 0 && nums[a] == nums[a - 1]) continue; // skip dup a
            int i = a + 1, j = nums.length - 1;
            while (i < j) {
                int sum = nums[a] + nums[i] + nums[j];
                if (sum == 0) {
                    result.add(Arrays.asList(nums[a], nums[i++], nums[j--]));
                    // dup-skip after a hit - see "Duplicate-skip Cheat Sheet" below
                    while (i < j && nums[i] == nums[i - 1]) i++; // skip dup i
                    while (i < j && nums[j] == nums[j + 1]) j--; // skip dup j
                } else if (sum < 0) {
                    i++;
                } else {
                    j--;
                }
            }
        }
        return result;
    }
}

```

Time $O(n^2)$ | **Space** $O(1)$ excluding output ($O(n)$ sort stack)

#18 4Sum

Description: Find all unique quadruplets summing to target.

Key: two nested fix loops + same opposite two pointer inner loop. Cast to `long` — sum of 4 ints can overflow.

Intuition: Pin two numbers, then the inner search is Two Sum — same idea as 3Sum with one more fixed level.

Variables: `a / b` = two fixed indices · `i / j` = inner left/right over `[b+1, n-1]` closed · `sum` = quadruplet sum (long to avoid overflow) **Pseudocode:**

```

sort the array
prepare the result list
let n be the length
for each first element a:
    skip a if it repeats the previous a
    for each second element b after a:
        skip b if it repeats the previous b
        set inner pointers after b and at the end
        while inner pointers haven't crossed:
            compute the four-way sum as a long
            if it equals target:
                record quadruplet and step both inner ends in
                skip duplicate left values
                skip duplicate right values
            else if too small:
                move left in
            else:
                move right in
return all quadruplets

```

```

class Solution {
    public List<List<Integer>> fourSum(int[] nums, int target) {
        Arrays.sort(nums);
        List<List<Integer>> result = new ArrayList<>();
        int n = nums.length;
        for (int a = 0; a < n - 3; a++) {
            if (a > 0 && nums[a] == nums[a - 1]) continue;
            for (int b = a + 1; b < n - 2; b++) {
                if (b > a + 1 && nums[b] == nums[b - 1]) continue;
                int i = b + 1, j = n - 1;
                while (i < j) {
                    long sum = (long) nums[a] + nums[b] + nums[i] + nums[j];
                    if (sum == target) {
                        result.add(Arrays.asList(nums[a], nums[b], nums[i++], nums[j--]));
                        // dup-skip after a hit - see "Duplicate-skip Cheat Sheet" below
                        while (i < j && nums[i] == nums[i - 1]) i++;
                        while (i < j && nums[j] == nums[j + 1]) j--;
                    } else if (sum < target) {
                        i++;
                    } else {
                        j--;
                    }
                }
            }
        }
        return result;
    }
}

```

Time $O(n^3)$ | **Space** $O(1)$ excluding output ($O(n)$ sort stack)

#11 Container With Most Water

Description: Given heights of vertical lines, find two lines forming the container with most water.

Key: always move the shorter side inward — moving the taller side can only make things worse.

Intuition: Width shrinks every step, so only a taller boundary can ever help; abandon the shorter wall.

Variables: i = left wall · j = right wall · max = best area, width is $j - i$ **Pseudocode:**

```

start walls at both ends, best area zero
while the walls haven't met:
    area = shorter wall times the width; keep the best
    if the left wall is shorter:
        abandon it, move left in
    else:
        abandon the right wall, move right in
return the best area

```

```

class Solution {
    public int maxArea(int[] height) {
        int i = 0, j = height.length - 1, max = 0;
        while (i < j) {
            max = Math.max(max, Math.min(height[i], height[j]) * (j - i));
            if (height[i] < height[j]) {
                i++;
            } else {
                j--;
            }
        }
        return max;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#42 Trapping Rain Water

Description: Given elevation heights, compute total water trapped after rain.

Key: water above position $x = \min(\text{maxLeft}, \text{maxRight}) - \text{height}[x]$. Track running max from each side.

Intuition: Water level at a cell is set by the shorter side's tallest wall, so always process from the lower side where that max is already known.

Variables: i / j = left/right scanners · $\text{maxI} / \text{maxJ}$ = tallest wall seen from each side · water = total trapped

Pseudocode:

```

start pointers at both ends, water and side-maxes at zero
while pointers haven't met:
    if the left side is the shorter wall:
        update the left max
        add water trapped above the left cell
        advance left
    else:
        update the right max
        add water trapped above the right cell
        advance right
return total water

```

```

class Solution {
    public int trap(int[] height) {
        int i = 0, j = height.length - 1, water = 0;
        int maxI = 0, maxJ = 0;
        while (i < j) {
            if (height[i] <= height[j]) {
                maxI = Math.max(maxI, height[i]);
                water += maxI - height[i];
                i++;
            } else {
                maxJ = Math.max(maxJ, height[j]);
                water += maxJ - height[j];
                j--;
            }
        }
        return water;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

Pattern 2 — Same Direction (Write Pointer)

Variables: `i` = write position (next free slot) · `j` = read position scanning every element **Pseudocode:**

```

start the writer at the front
let the reader scan every element:
    if this element passes the keep test,
        copy it to the write slot and advance the writer
return the new length (writer position)

```

```

// a fast reader scans everything; a slow writer trails behind and only copies the keepers. - WHEN: fi
int i = 0; // i = write position
for (int j = 0; j < n; j++) { // j = read position
    if (keepCondition(nums[j]))
        nums[i++] = nums[j];
}
return i;

```

`i` marks where the next valid element goes. `j` scans every element. Only write when the element passes the keep condition.

The generalised duplicate-skip pattern for "at most k duplicates":

Variables: `i` = write head · `j` = read scanner · `k` = how many duplicates are allowed **Pseudocode:**

```

start the writer at the front
scan every element with the reader:
    keep it if fewer than k written, or it differs from k slots back
    copy it and advance the writer
return the new length

```

```

int i = 0;
for (int j = 0; j < n; j++)
    if (i < k || nums[j] != nums[i - k]) // compare write head to k slots back
        nums[i++] = nums[j];
return i;

```

- $k = 1 \rightarrow \#26$ (no duplicates)
- $k = 2 \rightarrow \#80$ (at most 2)

#27 Remove Element

Description: Remove all occurrences of `val` in-place. Return new length.

Keep condition: `nums[j] != val`

Intuition: The writer only advances on keepers, so it always points at the next free slot for a kept value.

Variables: `i` = write position for kept values · `j` = read scanner **Pseudocode:**

```
start the writer at zero
scan each element with the reader:
    if it isn't the value to remove, write it and advance the writer
return the count of kept elements
```

```
class Solution {
    public int removeElement(int[] nums, int val) {
        int i = 0;
        for (int j = 0; j < nums.length; j++)
            if (nums[j] != val) nums[i++] = nums[j];
        return i;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#26 Remove Duplicates from Sorted Array

Description: Remove duplicates in-place so each element appears at most once. Return new length.

Keep condition: `nums[j] != nums[i-1]` — don't write if same as last written.

Intuition: Since the array is sorted, comparing against the last written value is enough to drop every duplicate run.

Variables: `i` = write position (also count of uniques) · `j` = read scanner **Pseudocode:**

```
start the writer at zero
scan each element:
    keep it if nothing written yet, or it differs from the last written value
    write it and advance the writer
return the unique count
```

```
class Solution {
    public int removeDuplicates(int[] nums) {
        int i = 0;
        for (int j = 0; j < nums.length; j++)
            if (i == 0 || nums[j] != nums[i - 1])
                nums[i++] = nums[j];
        return i;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#80 Remove Duplicates from Sorted Array II

Description: Each element may appear at most twice in-place. Return new length.

Keep condition: `nums[j] != nums[i-2]` — don't write if same as two slots back.

Intuition: Looking two slots back lets at most two copies through before the third is rejected.

Variables: `i` = write position (also kept count) · `j` = read scanner; compares to value two slots back **Pseudocode:**

```
start the writer at zero
scan each element:
    keep it if fewer than two written, or it differs from two slots back
    write it and advance the writer
return the kept count
```

```
class Solution {
    public int removeDuplicates(int[] nums) {
        int i = 0;
        for (int j = 0; j < nums.length; j++)
            if (i < 2 || nums[j] != nums[i - 2])
                nums[i++] = nums[j];
        return i;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#283 Move Zeroes

Description: Move all zeroes to end while preserving relative order of non-zero elements.

Keep condition: `nums[j] != 0` . Fill tail with zeros after the write pass.

Intuition: Compact the non-zeros to the front in order, then pad the leftover tail with zeros.

Variables: `i` = write position for non-zeros · `j` = read scanner **Pseudocode:**

```
start the writer at zero
scan each element:
    if non-zero, write it to the front and advance the writer
fill the remaining tail with zeros
```

```
class Solution {
    public void moveZeroes(int[] nums) {
        int i = 0;
        for (int j = 0; j < nums.length; j++)
            if (nums[j] != 0) nums[i++] = nums[j];
        while (i < nums.length) nums[i++] = 0;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#75 Sort Colors

Description: Sort array of 0s, 1s, 2s in-place without library sort.

Key: Dutch Flag with `LO = 0`, `MID = 1`, `HI = 2`.

Intuition: A single scan grows a "small" region from the left and a "large" region from the right, leaving mids in the middle.

Variables: `i` = boundary of the 0-region (low) · `k` = current scanner (mid) · `j` = boundary of the 2-region (high)

Pseudocode:

```
low boundary at start, scanner at start, high boundary at end
while the scanner hasn't passed the high boundary:
  if it's a 0:
    swap it down to the low region, advance both low and scanner
  else if it's a 1:
    leave it, just advance the scanner
  else (a 2):
    swap it up to the high region, shrink high (don't advance scanner)

helper: swap two array slots
```

```
class Solution {
  public void sortColors(int[] nums) {
    int i = 0, k = 0, j = nums.length - 1;
    while (k <= j) {
      if (nums[k] == 0) {
        swap(nums, i++, k++);
      } else if (nums[k] == 1) {
        k++;
      } else {
        swap(nums, k, j--);
      }
    }
  }
  private void swap(int[] a, int x, int y) {
    int t = a[x]; a[x] = a[y]; a[y] = t;
  }
}
```

Time O(n) | **Space** O(1)

Pattern Comparison

Pattern	Pointers move	Sort needed	Use when
Opposite	<code>i</code> right, <code>j</code> left, converge	Yes (usually)	Sum/target problems on sorted array
Same direction	<code>j</code> scans, <code>i</code> writes	No	Filter / compact array in-place
Dutch Flag	<code>k</code> scans, <code>i</code> low boundary, <code>j</code> high boundary	No	3-way partition (< / = / >)

Duplicate-skip Cheat Sheet (for Opposite pointers after a hit)

Variables: `i` / `j` = opposite pointers after recording a hit; advanced past equal neighbors **Pseudocode:**

```
after recording a result, step both ends inward
skip any left values equal to the one just used
skip any right values equal to the one just used
```

```
// After recording a result at (i, j):
i++; j--;
while (i < j && nums[i] == nums[i - 1]) i++; // skip duplicate i
while (i < j && nums[j] == nums[j + 1]) j--; // skip duplicate j
```

Always check `i < j` inside the skip loop to avoid crossing.

Additional Reference Problems

#1 Two Sum

Description: Given an integer array and a target sum, return the indices of two numbers that add up to target. Exactly one solution exists.

Key: hash map from value to index; for each element check if `target - x` was seen.

Intuition: Trade space for time — remember every value you've passed so the complement is a single lookup.

Variables: `seen` = map from value to its index · `i` = current scan index · `need` = complement `target - nums[i]`

Pseudocode:

```
create an empty value-to-index map
for each index i:
    compute the complement we need
    if the complement was already seen:
        return its index paired with i
    record the current value at i
return sentinel if no pair
```

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> seen = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int need = target - nums[i];
            if (seen.containsKey(need)) {
                return new int[]{seen.get(need), i};
            }
            seen.put(nums[i], i);
        }
        return new int[]{-1, -1};
    }
}
```

Time $O(n)$ | **Space** $O(n)$

#16 3Sum Closest

Description: Find three integers whose sum is closest to target. Return that sum.

Key: sort, fix one, opposite two pointers; track the closest sum seen.

Intuition: Same pin-and-sweep as 3Sum, but instead of matching exactly you keep the running closest.

Variables: `a` = fixed first index · `i / j` = inner left/right over `[a+1, n-1]` · `best` = closest sum so far · `sum` = current triplet sum **Pseudocode:**

```

sort the array
seed best with the first three elements
for each first element a:
    set inner pointers after a and at the end
    while they haven't crossed:
        compute the triplet sum
        if it is closer to target than best:
            update best
        if it exactly hits target:
            return it (cannot do better)
        else if too small:
            move left in
        else:
            move right in
return the closest sum

```

```

class Solution {
    public int threeSumClosest(int[] nums, int target) {
        Arrays.sort(nums);
        int best = nums[0] + nums[1] + nums[2];
        for (int a = 0; a < nums.length - 2; a++) {
            int i = a + 1, j = nums.length - 1;
            while (i < j) {
                int sum = nums[a] + nums[i] + nums[j];
                if (Math.abs(sum - target) < Math.abs(best - target)) {
                    best = sum;
                }
                if (sum == target) {
                    return sum;
                } else if (sum < target) {
                    i++;
                } else {
                    j--;
                }
            }
        }
        return best;
    }
}

```

Time $O(n^2)$ | **Space** $O(1)$

#31 Next Permutation

Description: Rearrange the array to its lexicographically next greater permutation in-place. If none, sort ascending.

Key: find first descending pivot from the right, swap with next larger, reverse the suffix.

Intuition: The longest decreasing suffix is already maximal; bump the digit just before it up to the smallest larger value, then reset the suffix to ascending.

Variables: n = length · i = pivot index (first descending from right) · j = smallest element larger than pivot in the suffix

Pseudocode:

```

let n be the length
start i one before the last
walk left while the suffix is non-increasing:
    step i left
if a pivot exists:
    scan from the right for the first element larger than the pivot
    walk j left while it's not larger:
        step j left
    swap pivot with that element
reverse the suffix after the pivot to make it ascending

helper: reverse a range
    swap inward until pointers cross

helper: swap two slots

```

```

class Solution {
    public void nextPermutation(int[] nums) {
        int n = nums.length;
        int i = n - 2;
        while (i >= 0 && nums[i] >= nums[i + 1]) {
            i--;
        }
        if (i >= 0) {
            int j = n - 1;
            while (nums[j] <= nums[i]) {
                j--;
            }
            swap(nums, i, j);
        }
        reverse(nums, i + 1, n - 1);
    }
    private void reverse(int[] a, int x, int y) {
        while (x < y) {
            swap(a, x++, y--);
        }
    }
    private void swap(int[] a, int x, int y) {
        int t = a[x]; a[x] = a[y]; a[y] = t;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#41 First Missing Positive

Description: Find the smallest missing positive integer in an unsorted array. $O(n)$ time, $O(1)$ space.

Key: place each value v in $[1, n]$ at index $v-1$ via swaps; first index where $nums[i] \neq i+1$ is the answer.

Intuition: Use the array itself as a hash table keyed by value, then scan for the first slot holding the wrong number.

Variables: n = length · i = scan/placement index · home slot of value v is index $v-1$ **Pseudocode:**

let n be the length
first pass, place each value at its home slot:
 while the current value is in [1,n] and not already home:
 swap it toward its correct slot v-1
second pass, find the first wrong slot:
 if slot i doesn't hold i+1, that's the missing positive
if all slots correct, answer is n+1

helper: swap two slots

```
class Solution {
    public int firstMissingPositive(int[] nums) {
        int n = nums.length;
        for (int i = 0; i < n; i++) {
            // place each value v in [1,n] at its home slot v-1 (cyclic swaps until it stays)
            while (nums[i] > 0 && nums[i] <= n && nums[nums[i] - 1] != nums[i]) {
                swap(nums, i, nums[i] - 1);
            }
        }
        for (int i = 0; i < n; i++) {
            if (nums[i] != i + 1) return i + 1; // first slot holding the wrong value
        }
        return n + 1;
    }
    private void swap(int[] a, int x, int y) {
        int t = a[x]; a[x] = a[y]; a[y] = t;
    }
}
```

Time O(n) | **Space** O(1)

#48 Rotate Image

Description: Rotate an n×n matrix 90 degrees clockwise in-place.

Key: transpose, then reverse each row.

Intuition: Transposing flips across the main diagonal; reversing each row then completes the clockwise turn.

Variables: n = matrix size · i / j = transpose row/col · lo / hi = row-reversal pointers **Pseudocode:**

```

let n be the matrix size
transpose: for each row i,
  for each column j above the diagonal,
    swap (i,j) with (j,i) to mirror across the diagonal

```

```

for each row, reverse it to finish the clockwise turn:
  set reversal pointers at both ends
  while they haven't crossed:
    swap the two ends

```

```

step left in
step right in

```

```

class Solution {
    public void rotate(int[][] matrix) {
        int n = matrix.length;
        for (int i = 0; i < n; i++) {           // transpose: mirror across the main diagonal
            for (int j = i + 1; j < n; j++) {
                int t = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = t;
            }
        }
        for (int i = 0; i < n; i++) {           // reverse each row → 90° clockwise
            int lo = 0, hi = n - 1;
            while (lo < hi) {
                int t = matrix[i][lo];
                matrix[i][lo] = matrix[i][hi];
                matrix[i][hi] = t;
                lo++;
                hi--;
            }
        }
    }
}

```

Time $O(n^2)$ | **Space** $O(1)$

#54 Spiral Matrix

Description: Return all elements of an $m \times n$ matrix in spiral order.

Key: maintain four shrinking boundaries (top, bottom, left, right); walk each edge then close in.

Intuition: Peel the matrix like an onion, one boundary layer per loop.

Variables: top / bottom / left / right = four shrinking layer boundaries · i / j = walk indices **Pseudocode:**

```

prepare the output list
set top/bottom boundaries
set left/right boundaries
while the box is still non-empty:
    walk the top edge left to right

    shrink the top down
    walk the right edge top to bottom

    shrink the right in
    if a bottom row remains:
        walk the bottom edge right to left

    shrink the bottom up
    if a left column remains:
        walk the left edge bottom to top

    shrink the left in
return the spiral

```

```

class Solution {
    public List<Integer> spiralOrder(int[][] matrix) {
        List<Integer> result = new ArrayList<>();
        int top = 0, bottom = matrix.length - 1;
        int left = 0, right = matrix[0].length - 1;
        while (top <= bottom && left <= right) {
            for (int j = left; j <= right; j++) {
                result.add(matrix[top][j]);
            }
            top++;
            for (int i = top; i <= bottom; i++) {
                result.add(matrix[i][right]);
            }
            right--;
            if (top <= bottom) {
                for (int j = right; j >= left; j--) {
                    result.add(matrix[bottom][j]);
                }
                bottom--;
            }
            if (left <= right) {
                for (int i = bottom; i >= top; i--) {
                    result.add(matrix[i][left]);
                }
                left++;
            }
        }
        return result;
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(1)$ excluding output

#73 Set Matrix Zeroes

Description: If any element is 0, set its entire row and column to 0, in-place.

Key: use first row/column as marker storage; handle their own zero state with two flags.

Intuition: Reuse the first row and column as the bookkeeping space so no extra arrays are needed.

Variables: m / n = dims · `firstRow / firstCol` = whether first row/col originally had a zero; first row/col reused as markers **Pseudocode:**

```
read dims
track whether first row and first col need zeroing
scan first row for a zero

    set the first-row flag

scan first col for a zero

    set the first-col flag

for each inner cell, if zero, mark its row/col headers:

    mark column header
    mark row header

for each inner cell, zero it if its header is marked:

    zero the cell

if first row was flagged, zero the whole first row

if first col was flagged, zero the whole first col
```

```

class Solution {
    public void setZeroes(int[][] matrix) {
        int m = matrix.length, n = matrix[0].length;
        boolean firstRow = false, firstCol = false;
        for (int j = 0; j < n; j++) {
            if (matrix[0][j] == 0) {
                firstRow = true;
            }
        }
        for (int i = 0; i < m; i++) {
            if (matrix[i][0] == 0) {
                firstCol = true;
            }
        }
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                if (matrix[i][j] == 0) {
                    matrix[i][0] = 0;
                    matrix[0][j] = 0;
                }
            }
        }
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                if (matrix[i][0] == 0 || matrix[0][j] == 0) {
                    matrix[i][j] = 0;
                }
            }
        }
        if (firstRow) {
            for (int j = 0; j < n; j++) {
                matrix[0][j] = 0;
            }
        }
        if (firstCol) {
            for (int i = 0; i < m; i++) {
                matrix[i][0] = 0;
            }
        }
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(1)$

#128 Longest Consecutive Sequence

Description: Find the length of the longest run of consecutive integers in an unsorted array. $O(n)$.

Key: put all in a set; only start counting from numbers with no predecessor ($x-1$ absent).

Intuition: Each sequence is counted exactly once by starting only at its smallest member.

Variables: `set` = all values · `x` = candidate sequence start · `cur / len` = current value and run length · `best` = longest run
Pseudocode:

put every value into a set

best length zero

for each value x:

only start a run if x has no predecessor:

walk forward counting consecutive successors

update the best length

return the best length

```
class Solution {
    public int longestConsecutive(int[] nums) {
        Set<Integer> set = new HashSet<>();
        for (int x : nums) {
            set.add(x);
        }
        int best = 0;
        for (int x : set) {
            if (!set.contains(x - 1)) {
                int cur = x, len = 1;
                while (set.contains(cur + 1)) {
                    cur++;
                    len++;
                }
                best = Math.max(best, len);
            }
        }
        return best;
    }
}
```

Time $O(n)$ | **Space** $O(n)$

#164 Maximum Gap

Description: Find the maximum gap between successive elements in the sorted form of an array. $O(n)$.

Key: bucket sort — the max gap is at least $\text{ceil}((\text{max}-\text{min})/(\text{n}-1))$; compare across adjacent non-empty buckets.

Intuition: With buckets sized at the minimum possible gap, the answer never lies inside a bucket, only between bucket boundaries.

Variables: n = length · min / max = extremes · $\text{bucketSize} / \text{bucketCount}$ = bucket geometry · $\text{bucketMin} / \text{bucketMax}$ = per-bucket extremes · prevMax = last non-empty bucket's max · gap = answer **Pseudocode:**

```
let n be the length
if fewer than two elements, gap is 0

find min and max

if all equal, gap is 0

bucket width = at least the average gap
bucket count covers the whole range
init each bucket's min/max

place each value into its bucket, updating bucket min/max

scan buckets left to right tracking previous max:

    skip empty buckets

    gap = max of cross-bucket gaps
    carry this bucket's max forward

return the max gap
```

```

class Solution {
    public int maximumGap(int[] nums) {
        int n = nums.length;
        if (n < 2) {
            return 0;
        }
        int min = Integer.MAX_VALUE, max = Integer.MIN_VALUE;
        for (int x : nums) {
            min = Math.min(min, x);
            max = Math.max(max, x);
        }
        if (min == max) {
            return 0;
        }
        int bucketSize = Math.max(1, (max - min) / (n - 1));
        int bucketCount = (max - min) / bucketSize + 1;
        int[] bucketMin = new int[bucketCount];
        int[] bucketMax = new int[bucketCount];
        Arrays.fill(bucketMin, Integer.MAX_VALUE);
        Arrays.fill(bucketMax, Integer.MIN_VALUE);
        for (int x : nums) {
            int b = (x - min) / bucketSize;
            bucketMin[b] = Math.min(bucketMin[b], x);
            bucketMax[b] = Math.max(bucketMax[b], x);
        }
        int gap = 0, prevMax = min;
        for (int b = 0; b < bucketCount; b++) {
            if (bucketMin[b] == Integer.MAX_VALUE) {
                continue;
            }
            gap = Math.max(gap, bucketMin[b] - prevMax);
            prevMax = bucketMax[b];
        }
        return gap;
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#169 Majority Element

Description: Find the element appearing more than $n/2$ times.

Key: Boyer-Moore voting — keep a candidate and a count; flip candidate when count hits 0.

Intuition: Pairing off different elements cancels them out; the majority element always survives.

Variables: `candidate` = current majority guess · `count` = its running vote balance · `x` = scanned value **Pseudocode:**

```

seed candidate, count zero
for each value x:
    if count hit zero, adopt x as the new candidate

    if x matches the candidate, add a vote

    else cancel a vote

return the surviving candidate

```

```

class Solution {
    public int majorityElement(int[] nums) {
        int candidate = nums[0], count = 0;
        for (int x : nums) {
            if (count == 0) {
                candidate = x;
            }
            if (x == candidate) {
                count++;
            } else {
                count--;
            }
        }
        return candidate;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#189 Rotate Array

Description: Rotate the array right by k positions, in-place.

Key: reverse whole array, then reverse first k and last $n-k$.

Intuition: Two partial reversals after a full reversal place each block in its rotated position without extra space.

Variables: n = length · k = effective shift $k \% n$; reverse helper uses i / j ends **Pseudocode:**

```

let n be the length
reduce k modulo n
reverse the whole array
reverse the first k elements
reverse the remaining tail

helper: reverse a range
    while ends haven't crossed:
        swap the two ends
        step left in
        step right in

```

```

class Solution {
    public void rotate(int[] nums, int k) {
        int n = nums.length;
        k %= n;
        reverse(nums, 0, n - 1);    // reverse all, then un-reverse each block into place
        reverse(nums, 0, k - 1);
        reverse(nums, k, n - 1);
    }
    private void reverse(int[] a, int i, int j) {
        while (i < j) {
            int t = a[i]; a[i] = a[j]; a[j] = t;
            i++;
            j--;
        }
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#217 Contains Duplicate

Description: Return true if any value appears at least twice.

Key: add to a set; if an insert fails, a duplicate exists.

Intuition: A set rejects repeats, so the first failed insert proves a duplicate.

Variables: `seen` = set of values seen · `x` = scanned value **Pseudocode:**

```

create an empty set
for each value x:
    if adding it fails (already present):
        a duplicate exists

no duplicate found

```

```

class Solution {
    public boolean containsDuplicate(int[] nums) {
        Set<Integer> seen = new HashSet<>();
        for (int x : nums) {
            if (!seen.add(x)) {
                return true;
            }
        }
        return false;
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#220 Contains Duplicate III

Description: Are there indices i, j with $|nums[i] - nums[j]| \leq \text{valueDiff}$ and $|i - j| \leq \text{indexDiff}$?

Key: bucket by value width ($\text{valueDiff} + 1$); only adjacent buckets can satisfy the value constraint.

Intuition: Same-bucket means automatically within range; neighbor buckets need an explicit check. Slide a window of size indexDiff .

Variables: `buckets` = map from bucket id to value · `width` = `valueDiff+1` · `id` = current value's bucket; window holds last `indexDiff` values **Pseudocode:**

```
if valueDiff is negative, impossible

map of bucket id to value
bucket width is valueDiff+1
for each index i:
    compute this value's bucket id
    if its bucket is occupied, a close pair exists

    if the lower neighbor bucket is within value range, done

    if the upper neighbor bucket is within value range, done

    store this value in its bucket
    if the window is full, evict the value leaving the window

no qualifying pair

helper: floor-divide value into a bucket id (handles negatives)
```

```
class Solution {
    public boolean containsNearbyAlmostDuplicate(int[] nums, int indexDiff, int valueDiff) {
        if (valueDiff < 0) {
            return false;
        }
        Map<Long, Long> buckets = new HashMap<>();
        long width = (long) valueDiff + 1;
        for (int i = 0; i < nums.length; i++) {
            long id = bucketId(nums[i], width);
            if (buckets.containsKey(id)) {
                return true;
            }
            if (buckets.containsKey(id - 1) && Math.abs(nums[i] - buckets.get(id - 1)) < width) {
                return true;
            }
            if (buckets.containsKey(id + 1) && Math.abs(nums[i] - buckets.get(id + 1)) < width) {
                return true;
            }
            buckets.put(id, (long) nums[i]);
            if (i >= indexDiff) {
                buckets.remove(bucketId(nums[i - indexDiff], width));
            }
        }
        return false;
    }
    private long bucketId(long x, long width) {
        return x < 0 ? (x + 1) / width - 1 : x / width;
    }
}
```

Time $O(n)$ | **Space** $O(\min(n, \text{indexDiff}))$

#229 Majority Element II

Description: Find all elements appearing more than $n/3$ times.

Key: Boyer-Moore with two candidates (at most two can exceed $n/3$); verify with a second pass.

Intuition: At most two elements can each occur more than a third of the time, so track two running candidates.

Variables: $c1 / c2$ = two candidates · $n1 / n2$ = their vote counts · x = scanned value **Pseudocode:**

```
init two candidates and zero counts
for each value x:
    if it's candidate one, bump its count

    else if candidate two, bump its count

    else if slot one is empty, adopt x there

    else if slot two is empty, adopt x there

    else cancel one vote from each

reset counts for a verification pass

recount actual occurrences

collect candidates exceeding  $n/3$ 

return the winners
```

```

class Solution {
    public List<Integer> majorityElement(int[] nums) {
        int c1 = 0, c2 = 1, n1 = 0, n2 = 0;
        for (int x : nums) {
            if (x == c1) {
                n1++;
            } else if (x == c2) {
                n2++;
            } else if (n1 == 0) {
                c1 = x;
                n1 = 1;
            } else if (n2 == 0) {
                c2 = x;
                n2 = 1;
            } else {
                n1--;
                n2--;
            }
        }
        n1 = 0;
        n2 = 0;
        for (int x : nums) {
            if (x == c1) {
                n1++;
            } else if (x == c2) {
                n2++;
            }
        }
        List<Integer> result = new ArrayList<>();
        if (n1 > nums.length / 3) {
            result.add(c1);
        }
        if (n2 > nums.length / 3) {
            result.add(c2);
        }
        return result;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#238 Product of Array Except Self

Description: Return output where `output[i]` is the product of all elements except `nums[i]` . No division.

Key: prefix products left-to-right, then multiply by suffix products right-to-left.

Intuition: Each position's answer is everything to its left times everything to its right.

Variables: `n` = length · `result` = output (left products then folded right) · `suffix` = running product to the right

Pseudocode:

```

let n be the length
allocate the output
seed first prefix product as 1
for i from 1: result[i] = product of everything to the left

running right product starts at 1
for i from the end down: fold in the right product

    advance the suffix product
return the result

```

```

class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        result[0] = 1;
        for (int i = 1; i < n; i++) {
            result[i] = result[i - 1] * nums[i - 1];    // result[i] = product of everything to the left
        }
        int suffix = 1;
        for (int i = n - 1; i >= 0; i--) {
            result[i] *= suffix;                        // then fold in the running product to the right
            suffix *= nums[i];
        }
        return result;
    }
}

```

Time $O(n)$ | **Space** $O(1)$ excluding output

#259 3Sum Smaller

Description: Count triplets with sum strictly less than target in a sorted array.

Key: fix one, opposite pointers; when `sum < target`, all pairs between `i` and `j` also qualify → add `j-i`.

Intuition: Once the largest partner works, every smaller partner does too, so count them in bulk.

Variables: `a` = fixed first index · `i / j` = inner left/right over `[a+1, n-1]` · `count` = qualifying triplets **Pseudocode:**

```

sort the array
count zero
for each first element a:
    set inner pointers after a and at the end
    while they haven't crossed:
        if the triplet sum is below target:
            every partner between i and j also qualifies, add j-i
            move left in
        else:
            move right in to shrink the sum
return the count

```

```

class Solution {
    public int threeSumSmaller(int[] nums, int target) {
        Arrays.sort(nums);
        int count = 0;
        for (int a = 0; a < nums.length - 2; a++) {
            int i = a + 1, j = nums.length - 1;
            while (i < j) {
                if (nums[a] + nums[i] + nums[j] < target) {
                    count += j - i;    // ← VARIATION: bulk-count all valid j for this i
                    i++;
                } else {
                    j--;
                }
            }
        }
        return count;
    }
}

```

Time $O(n^2)$ | **Space** $O(1)$

#280 Wiggle Sort

Description: Reorder in-place so `nums[0] <= nums[1] >= nums[2] <= nums[3]...`

Key: single pass; fix each adjacent pair that violates the expected up/down relation by swapping.

Intuition: A greedy local swap is always safe — it never breaks the pair you already fixed.

Variables: `i` = pair index; even `i` expects a rise, odd expects a fall **Pseudocode:**

```

for each adjacent pair index i:
    decide whether this position should rise
    if the actual relation violates the expectation:
        swap the pair to fix it

```

```

class Solution {
    public void wiggleSort(int[] nums) {
        for (int i = 0; i < nums.length - 1; i++) {
            boolean shouldRise = (i % 2 == 0);
            if (shouldRise == (nums[i] > nums[i + 1])) {
                int t = nums[i];
                nums[i] = nums[i + 1];
                nums[i + 1] = t;
            }
        }
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#289 Game of Life

Description: Simulate one Game of Life step, updating all cells simultaneously, in-place.

Key: encode next state in the second bit (e.g. `01` → `11` for live→live); shift right at the end.

Intuition: Pack the new state into a spare bit so neighbor counts still read the old state during the pass.

Variables: `m / n` = dims · `dr / dc` = 8 neighbor offsets · `live` = live neighbor count; bit 1 = old state, bit 2 = new

Pseudocode:

```
read dims
neighbor row offsets
neighbor col offsets
for each cell (r,c):

    count live neighbors using the old (low) bit:

    if currently alive:
        survives on 2 or 3 neighbors: set the new-state bit

    else (dead):
        becomes alive on exactly 3: set the new-state bit

second pass: shift each cell to expose the new state
```

```

class Solution {
    public void gameOfLife(int[][] board) {
        int m = board.length, n = board[0].length;
        int[] dr = {1, -1, 0, 0, 1, 1, -1, -1};
        int[] dc = {0, 0, 1, -1, 1, -1, 1, -1};
        for (int r = 0; r < m; r++) {
            for (int c = 0; c < n; c++) {
                int live = 0;
                for (int dir = 0; dir < 8; dir++) {
                    int nr = r + dr[dir], nc = c + dc[dir];
                    if (nr >= 0 && nr < m && nc >= 0 && nc < n && (board[nr][nc] & 1) == 1) {
                        live++;
                    }
                }
                if ((board[r][c] & 1) == 1) {
                    if (live == 2 || live == 3) {
                        board[r][c] |= 2;
                    }
                } else {
                    if (live == 3) {
                        board[r][c] |= 2;
                    }
                }
            }
        }
        for (int r = 0; r < m; r++) {
            for (int c = 0; c < n; c++) {
                board[r][c] >>= 1;
            }
        }
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(1)$

#303 Range Sum Query - Immutable

Description: Answer many range-sum queries on a static array efficiently.

Key: precompute prefix sums; $\text{sumRange}(i, j) = \text{prefix}[j+1] - \text{prefix}[i]$.

Intuition: A range sum is the difference of two prefix sums, so each query is $O(1)$ after one build pass.

Variables: `prefix` = prefix sums with `prefix[0]=0`; query range `[i, j]` closed **Pseudocode:**

```

store a prefix-sum array
constructor builds prefix sums:
    size one larger than input
    for each element, prefix[i+1] = prefix[i] + value

```

```

query: range sum is prefix[j+1] - prefix[i]

```

```

class NumArray {
    private int[] prefix;
    public NumArray(int[] nums) {
        prefix = new int[nums.length + 1];
        for (int i = 0; i < nums.length; i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }
    }
    public int sumRange(int i, int j) {
        return prefix[j + 1] - prefix[i];
    }
}

```

Time $O(n)$ build, $O(1)$ query | **Space** $O(n)$

#304 Range Sum Query 2D - Immutable

Description: Answer many 2D region-sum queries on a static matrix efficiently.

Key: 2D prefix sums with inclusion-exclusion.

Intuition: Each region sum combines four corner prefix sums via inclusion-exclusion.

Variables: m / n = dims · `prefix` = 2D prefix sums padded by one; region `[row1,col1]..[row2,col2]` closed

Pseudocode:

```

store a 2D prefix array
constructor builds it:
    read dims
    allocate padded prefix grid
    for each cell, combine top, left, minus overlap, plus value:

```

```

query: inclusion-exclusion of four corners

```

```

class NumMatrix {
    private int[][] prefix;
    public NumMatrix(int[][] matrix) {
        int m = matrix.length, n = matrix[0].length;
        prefix = new int[m + 1][n + 1];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                prefix[i + 1][j + 1] = matrix[i][j] + prefix[i][j + 1]
                    + prefix[i + 1][j] - prefix[i][j];
            }
        }
    }
    public int sumRegion(int row1, int col1, int row2, int col2) {
        return prefix[row2 + 1][col2 + 1] - prefix[row1][col2 + 1]
            - prefix[row2 + 1][col1] + prefix[row1][col1];
    }
}

```

Time $O(mn)$ build, $O(1)$ query | **Space** $O(mn)$

#307 Range Sum Query - Mutable

Description: Support range-sum queries and point updates.

Key: Binary Indexed Tree (Fenwick) — both update and prefix query in $O(\log n)$.

Intuition: A Fenwick tree stores partial sums over power-of-two ranges so updates and queries each touch only $\log n$ nodes.

Variables: `tree` = Fenwick array (1-indexed) · `nums` = current values · `n` = size; `i & (-i)` = lowest set bit step

Pseudocode:

```
fields: tree, values, size
constructor:
  store size
  allocate value store
  allocate Fenwick tree
  seed by updating each index

update: apply delta up the tree
  compute delta from old value
  store the new value
  climb adding delta at each parent

range sum: prefix(right+1) - prefix(left)

prefix helper: walk down summing

  while index positive:
    add this node
    drop the lowest set bit

  return the sum
```



```

class NumArray {
    private int[] tree;
    private int[] nums;
    private int n;
    public NumArray(int[] nums) {
        this.n = nums.length;
        this.nums = new int[n];
        this.tree = new int[n + 1];
        for (int i = 0; i < n; i++) {
            update(i, nums[i]);
        }
    }
    public void update(int index, int val) {
        int delta = val - nums[index];
        nums[index] = val;
        for (int i = index + 1; i <= n; i += i & (-i)) {
            tree[i] += delta;
        }
    }
    public int sumRange(int left, int right) {
        return prefix(right + 1) - prefix(left);
    }
    private int prefix(int i) {
        int s = 0;
        while (i > 0) {
            s += tree[i];
            i -= i & (-i);
        }
        return s;
    }
}

```

Time $O(\log n)$ update/query | **Space** $O(n)$

#311 Sparse Matrix Multiplication

Description: Multiply two sparse matrices, skipping zero elements.

Key: iterate only over non-zero entries of the first matrix; for each, fan out across the matching row of the second.

Intuition: A zero in A contributes nothing, so skip it entirely instead of doing a full triple loop.

Variables: $m / k / n$ = dims · $i / p / j$ = row of A / shared dim / col of B; skip zeros in A **Pseudocode:**

```

read dims
allocate the result
for each row i of A:
    for each shared index p:
        if A[i][p] is zero,
            skip it (contributes nothing)
        for each col j of B:
            if B[p][j] is non-zero,
                accumulate the product

return the product matrix

```

```

class Solution {
    public int[][] multiply(int[][] mat1, int[][] mat2) {
        int m = mat1.length, k = mat1[0].length, n = mat2[0].length;
        int[][] result = new int[m][n];
        for (int i = 0; i < m; i++) {
            for (int p = 0; p < k; p++) {
                if (mat1[i][p] == 0) {
                    continue; // ← VARIATION: skip zero to exploit sparsity
                }
                for (int j = 0; j < n; j++) {
                    if (mat2[p][j] != 0) {
                        result[i][j] += mat1[i][p] * mat2[p][j];
                    }
                }
            }
        }
        return result;
    }
}

```

Time $O(mkn)$ worst case | **Space** $O(m \cdot n)$ excluding output

#315 Count of Smaller Numbers After Self

Description: For each element, count elements to its right that are smaller. $O(n \log n)$.

Key: merge sort on (value, index) pairs; while merging, count how many right-half elements slot before a left element.

Intuition: During a merge, every time a right element is taken before a left one, it is a smaller-to-the-right for that left element.

Variables: `counts` = answer per original index · `indices` = index array sorted by value · `i / j` = merge pointers · `rightCount` = right-half elements already placed before current left **Pseudocode:**

field: counts per index

set up:

- let n be the length

- allocate counts

- index array

- fill with identity indices

- run merge sort over indices

- build the result list from counts

- return it

merge sort:

- base case single element

- midpoint

- sort left half

- sort right half

- merge buffer and counters

- while both halves have elements:

- if the right value is smaller:

- it jumps ahead of remaining left items, bump rightCount

- take the right index

- else:

- credit accumulated rightCount to this left element

- take the left index

- drain the left half, crediting rightCount

- drain the right half

- copy merged order back

```

class Solution {
    private int[] counts;
    public List<Integer> countSmaller(int[] nums) {
        int n = nums.length;
        counts = new int[n];
        int[] indices = new int[n];
        for (int i = 0; i < n; i++) {
            indices[i] = i;
        }
        mergeSort(nums, indices, 0, n - 1);
        List<Integer> result = new ArrayList<>();
        for (int c : counts) {
            result.add(c);
        }
        return result;
    }
    private void mergeSort(int[] nums, int[] idx, int lo, int hi) {
        if (lo >= hi) {
            return;
        }
        int mid = (lo + hi) >> 1;
        mergeSort(nums, idx, lo, mid);
        mergeSort(nums, idx, mid + 1, hi);
        int[] merged = new int[hi - lo + 1];
        int i = lo, j = mid + 1, k = 0, rightCount = 0;
        while (i <= mid && j <= hi) {
            if (nums[idx[j]] < nums[idx[i]]) {
                rightCount++;
                merged[k++] = idx[j++];
            } else {
                counts[idx[i]] += rightCount;
                merged[k++] = idx[i++];
            }
        }
        while (i <= mid) {
            counts[idx[i]] += rightCount;
            merged[k++] = idx[i++];
        }
        while (j <= hi) {
            merged[k++] = idx[j++];
        }
        System.arraycopy(merged, 0, idx, lo, merged.length);
    }
}

```

Time $O(n \log n)$ | **Space** $O(n)$

#325 Maximum Size Subarray Sum Equals k

Description: Find the longest subarray with sum equal to k. $O(n)$.

Key: prefix sum + hash map of earliest index for each prefix; for current sum s , look for $s - k$.

Intuition: A subarray sums to k exactly when two prefix sums differ by k; keep the earliest index to maximize length.

Variables: `firstIndex` = map prefix sum to its earliest index · `sum` = running prefix · `best` = longest length

Pseudocode:

```
map of prefix sum to earliest index
seed sum 0 at index -1
running sum and best length
for each index i:
    add the element to the running sum
    if some earlier prefix equals sum-k:
        candidate length is i minus that earliest index
    record this prefix only if new (keep earliest)
return the best length
```

```
class Solution {
    public int maxSubArrayLen(int[] nums, int k) {
        Map<Integer, Integer> firstIndex = new HashMap<>();
        firstIndex.put(0, -1);
        int sum = 0, best = 0;
        for (int i = 0; i < nums.length; i++) {
            sum += nums[i];
            if (firstIndex.containsKey(sum - k)) {
                best = Math.max(best, i - firstIndex.get(sum - k));
            }
            firstIndex.putIfAbsent(sum, i);
        }
        return best;
    }
}
```

Time $O(n)$ | **Space** $O(n)$

#327 Count of Range Sum

Description: Count range sums that lie in $[lower, upper]$. $O(n \log n)$.

Key: prefix sums + merge sort; while merging count prefix pairs whose difference falls in range.

Intuition: A range sum is a difference of two prefix sums; merge sort lets you count qualifying pairs across halves efficiently.

Variables: `lower / upper` = bounds · `prefix` = prefix sums · `low / high` = sliding bounds in right half counting valid differences · `i / j` = merge pointers **Pseudocode:**

store the bounds

build prefix sums (one longer)

recurse and count

merge-count:

base case

midpoint

count from both halves

two moving bounds in the right half

for each left prefix i:

advance low past differences below lower

advance high past differences within upper

add the count of qualifying right prefixes

merge the two sorted halves:

while both have elements:

take smaller left

take smaller right

drain left

drain right

copy back

return the count

```

class Solution {
    private long lower, upper;
    public int countRangeSum(int[] nums, int lower, int upper) {
        this.lower = lower;
        this.upper = upper;
        long[] prefix = new long[nums.length + 1];
        for (int i = 0; i < nums.length; i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }
        return mergeCount(prefix, 0, prefix.length - 1);
    }
    private int mergeCount(long[] prefix, int lo, int hi) {
        if (lo >= hi) {
            return 0;
        }
        int mid = (lo + hi) >> 1;
        int count = mergeCount(prefix, lo, mid) + mergeCount(prefix, mid + 1, hi);
        int low = mid + 1, high = mid + 1;
        for (int i = lo; i <= mid; i++) {
            while (low <= hi && prefix[low] - prefix[i] < lower) {
                low++;
            }
            while (high <= hi && prefix[high] - prefix[i] <= upper) {
                high++;
            }
            count += high - low;
        }
        long[] merged = new long[hi - lo + 1];
        int i = lo, j = mid + 1, k = 0;
        while (i <= mid && j <= hi) {
            if (prefix[i] <= prefix[j]) {
                merged[k++] = prefix[i++];
            } else {
                merged[k++] = prefix[j++];
            }
        }
        while (i <= mid) {
            merged[k++] = prefix[i++];
        }
        while (j <= hi) {
            merged[k++] = prefix[j++];
        }
        System.arraycopy(merged, 0, prefix, lo, merged.length);
        return count;
    }
}

```

Time $O(n \log n)$ | **Space** $O(n)$

#334 Increasing Triplet Subsequence

Description: Return true if there exists $i < j < k$ with $nums[i] < nums[j] < nums[k]$.

Key: track smallest (first) and smallest second value (second); any element beating second completes the triplet.

Intuition: Maintaining the two smallest ascending values seen so far is enough to detect a third larger one.

Variables: first = smallest seen · second = smallest value that has a smaller predecessor · x = scanned value

Pseudocode:

```
init first and second to +infinity
for each value x:
    if it's a new smallest, update first

    else if it can be a new second smallest, update second

    else it beats second, so a triplet exists

no triplet found
```

```
class Solution {
    public boolean increasingTriplet(int[] nums) {
        int first = Integer.MAX_VALUE, second = Integer.MAX_VALUE;
        for (int x : nums) {
            if (x <= first) {
                first = x;
            } else if (x <= second) {
                second = x;
            } else {
                return true;
            }
        }
        return false;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#349 Intersection of Two Arrays

Description: Return the intersection of two arrays (unique elements only).

Key: put one array in a set; collect elements of the other that are present.

Intuition: Set membership turns intersection into linear lookups.

Variables: `set` = values of `nums1` · `common` = intersection set · `result` = output array · `i` = fill index **Pseudocode:**

```
put all of nums1 into a set

collect shared values without repeats

copy the set into an array

return it
```



```

class Solution {
    public int[] intersection(int[] nums1, int[] nums2) {
        Set<Integer> set = new HashSet<>();
        for (int x : nums1) {
            set.add(x);
        }
        Set<Integer> common = new HashSet<>();
        for (int x : nums2) {
            if (set.contains(x)) {
                common.add(x);
            }
        }
        int[] result = new int[common.size()];
        int i = 0;
        for (int x : common) {
            result[i++] = x;
        }
        return result;
    }
}

```

Time $O(n + m)$ | **Space** $O(n)$

#350 Intersection of Two Arrays II

Description: Return the intersection including duplicates (multiplicity is the min of counts).

Key: count one array in a map; for the other, emit and decrement while count > 0.

Intuition: A frequency map lets each shared occurrence be matched at most once.

Variables: `count` = frequency map of `nums1` · `result` = matched values · `arr` = output array **Pseudocode:**

count each value of `nums1`

for each value in `nums2`:

 if its count is still positive:

 emit it

 decrement its count

copy results to an array

return it

```

class Solution {
    public int[] intersect(int[] nums1, int[] nums2) {
        Map<Integer, Integer> count = new HashMap<>();
        for (int x : nums1) {
            count.merge(x, 1, Integer::sum);
        }
        List<Integer> result = new ArrayList<>();
        for (int x : nums2) {
            if (count.getOrDefault(x, 0) > 0) {
                result.add(x);
                count.put(x, count.get(x) - 1);
            }
        }
        int[] arr = new int[result.size()];
        for (int i = 0; i < arr.length; i++) {
            arr[i] = result.get(i);
        }
        return arr;
    }
}

```

Time $O(n + m)$ | **Space** $O(\min(n, m))$

#370 Range Addition

Description: Apply range updates $[start, end] += inc$ then return the final array.

Key: difference array — add inc at $start$, subtract at $end+1$, then prefix-sum.

Intuition: Mark only the endpoints of each update; a single prefix pass materializes all increments.

Variables: `diff` = difference array (one longer) · `running` = prefix accumulator · `result` = materialized array

Pseudocode:

```

allocate a difference array
for each update [start,end,inc]:
    add inc at start
    subtract inc just past end
allocate the result
running total zero
prefix-sum the diff into the result:
    add this slot's delta
    store the running value
return the result

```

```

class Solution {
    public int[] getModifiedArray(int length, int[][] updates) {
        int[] diff = new int[length + 1];
        for (int[] u : updates) {
            diff[u[0]] += u[2];
            diff[u[1] + 1] -= u[2];
        }
        int[] result = new int[length];
        int running = 0;
        for (int i = 0; i < length; i++) {
            running += diff[i];
            result[i] = running;
        }
        return result;
    }
}

```

Time $O(n + \text{updates})$ | **Space** $O(n)$

#384 Shuffle an Array

Description: Return a uniformly random permutation of the array; support reset to original.

Key: Fisher-Yates — for each i swap with a random index in $[i, n)$.

Intuition: Choosing each position's element uniformly from the remaining ones yields every permutation with equal probability.

Variables: `original` = pristine copy · `arr` = working copy · `rng` = randomness · `i / j` = Fisher-Yates swap indices

Pseudocode:

```

fields: original, working array, RNG
constructor: keep a pristine copy and a working copy

reset: restore from the pristine copy

shuffle: Fisher-Yates from the back
    for i from the end down to 1:
        pick a random j in [0, i]
        swap positions i and j

return the shuffled array

```

```

class Solution {
    private int[] original;
    private int[] arr;
    private Random rng = new Random();
    public Solution(int[] nums) {
        original = nums.clone();
        arr = nums.clone();
    }
    public int[] reset() {
        arr = original.clone();
        return arr;
    }
    public int[] shuffle() {
        for (int i = arr.length - 1; i > 0; i--) {
            int j = rng.nextInt(i + 1);
            int t = arr[i]; arr[i] = arr[j]; arr[j] = t;
        }
        return arr;
    }
}

```

Time $O(n)$ per shuffle | **Space** $O(n)$

#414 Third Maximum Number

Description: Return the third distinct maximum; if it doesn't exist, return the maximum.

Key: track three distinct maxima with Long sentinels to handle Integer.MIN_VALUE.

Intuition: Keep a sorted top-three as you scan, skipping duplicates.

Variables: `first` / `second` / `third` = top three distinct maxima (Long so MIN_VALUE works) · `v` = boxed current value

Pseudocode:

```

three distinct-max holders, all null
for each value x:
    box it
    skip if it duplicates any current holder
    if it's a new overall max, cascade the top three down

    else if it's a new second, cascade second/third

    else if it's a new third, set third

return the third if it exists, otherwise the max

```

```

class Solution {
    public int thirdMax(int[] nums) {
        Long first = null, second = null, third = null;
        for (int x : nums) {
            Long v = (long) x;
            if (v.equals(first) || v.equals(second) || v.equals(third)) continue; // skip duplicates
            if (first == null || v > first) { // new max → cascade the old top-three down
                third = second; second = first; first = v;
            } else if (second == null || v > second) {
                third = second; second = v;
            } else if (third == null || v > third) {
                third = v;
            }
        }
        return (int) (long) (third == null ? first : third); // no 3rd distinct → return the max
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#419 Battleships in a Board

Description: Count battleships ('X' runs, horizontal or vertical, never adjacent).

Key: count only the top-left cell of each ship — an 'X' with no 'X' directly above or left.

Intuition: Each ship has exactly one cell with no ship-neighbor above or to its left, so count those.

Variables: `count` = ship count · `r / c` = scan cell; count only a ship's top-left cell **Pseudocode:**

```

count zero
for each row r:
    for each col c:
        if this cell is a ship AND
           has no ship directly above AND
           has no ship directly to the left:
               it's a new ship, count it

return the count

```

```

class Solution {
    public int countBattleships(char[][] board) {
        int count = 0;
        for (int r = 0; r < board.length; r++) {
            for (int c = 0; c < board[0].length; c++) {
                if (board[r][c] == 'X'
                    && (r == 0 || board[r - 1][c] != 'X')
                    && (c == 0 || board[r][c - 1] != 'X')) {
                    count++;
                }
            }
        }
        return count;
    }
}

```

Time $O(m*n)$ | **Space** $O(1)$

#442 Find All Duplicates in an Array

Description: Values in $[1, n]$, each appears once or twice. Return the ones appearing twice. $O(n)$ time, $O(1)$ extra space.

Key: use the sign at index $|x|-1$ as a "seen" marker; a second visit finds it already negative.

Intuition: Flip the sign at each value's home index; encountering an already-negative slot means that value repeated.

Variables: `result` = duplicates found · `x` = scanned value · `idx` = home index $|x|-1$ used as a seen-marker via sign

Pseudocode:

```
prepare the result list
for each value x:
    compute its home index from absolute value
    if that slot is already negative:
        we've seen this value before, record it
    else:
        flip the sign to mark it seen

return the duplicates
```

```
class Solution {
    public List<Integer> findDuplicates(int[] nums) {
        List<Integer> result = new ArrayList<>();
        for (int x : nums) {
            int idx = Math.abs(x) - 1;
            if (nums[idx] < 0) {
                result.add(idx + 1);
            } else {
                nums[idx] = -nums[idx];
            }
        }
        return result;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#448 Find All Numbers Disappeared in an Array

Description: Values in $[1, n]$. Return all numbers in $[1, n]$ missing from the array.

Key: mark index $|x|-1$ negative; positive slots correspond to missing numbers.

Intuition: Use sign marking; any home index never marked means that number was absent.

Variables: `x` = scanned value · `idx` = home index $|x|-1$; unmarked (positive) slots reveal missing numbers · `result` = answer **Pseudocode:**

```

for each value x:
    compute its home index
    if that slot is still positive:
        mark it negative as present

prepare the result list
scan all slots:
    any still-positive slot means that number is missing

return the missing numbers

```

```

class Solution {
    public List<Integer> findDisappearedNumbers(int[] nums) {
        for (int x : nums) {
            int idx = Math.abs(x) - 1;
            if (nums[idx] > 0) {
                nums[idx] = -nums[idx];
            }
        }
        List<Integer> result = new ArrayList<>();
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] > 0) {
                result.add(i + 1);
            }
        }
        return result;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#454 4Sum II

Description: Count quadruplets (i, j, k, l) with $A[i] + B[j] + C[k] + D[l] == 0$.

Key: map all sums of A+B; for each C+D look up $-(c+d)$.

Intuition: Split four arrays into two pairs so a hash of one pair's sums turns the search into $O(n^2)$ lookups.

Variables: `abSum` = map from A+B pair-sum to its frequency · `count` = matching quadruplets **Pseudocode:**

```

map of A+B sums to counts
for each a, for each b:

    tally the sum a+b

count zero
for each c, for each d:

    add how many A+B sums equal  $-(c+d)$ 

return the count

```

```

class Solution {
    public int fourSumCount(int[] a, int[] b, int[] c, int[] d) {
        Map<Integer, Integer> abSum = new HashMap<>();
        for (int x : a) {
            for (int y : b) {
                abSum.merge(x + y, 1, Integer::sum);
            }
        }
        int count = 0;
        for (int x : c) {
            for (int y : d) {
                count += abSum.getOrDefault(-(x + y), 0);
            }
        }
        return count;
    }
}

```

Time $O(n^2)$ | **Space** $O(n^2)$

#457 Circular Array Loop

Description: Detect a cycle of length > 1 with consistent direction in a circular array of jumps.

Key: Floyd's tortoise and hare; movement direction must stay consistent and the cycle length must exceed 1.

Intuition: Fast/slow pointers detect cycles; reject single-element self-loops and direction flips.

Variables: n = length · `slow` / `fast` = Floyd tortoise/hare · `forward` = direction of this run; `next()` returns -1 on flip or self-loop **Pseudocode:**

store the length

for each start index i :
 record this run's direction
 place both pointers at i

loop advancing slow once, fast twice:
 step slow
 step fast once
 step fast again if still valid
 if either invalidated, abandon this start
 if they meet, a valid cycle exists

no valid loop

`next()`: given index and direction

if the direction flipped here, invalid
 compute the wrapped target index
 reject a single-element self-loop


```

class Solution {
    private int n;
    public boolean circularArrayLoop(int[] nums) {
        n = nums.length;
        for (int i = 0; i < n; i++) {
            boolean forward = nums[i] > 0;
            int slow = i, fast = i;
            // advance hare twice and tortoise once; next() returns -1 on a direction flip or self-loop
            while (true) {
                slow = next(nums, slow, forward);
                fast = next(nums, fast, forward);
                if (fast != -1) fast = next(nums, fast, forward);
                if (slow == -1 || fast == -1) break;
                if (slow == fast) return true; // pointers meet → valid cycle of length > 1
            }
        }
        return false;
    }
    private int next(int[] nums, int i, boolean forward) {
        if ((nums[i] > 0) != forward) return -1; // direction flipped → not this cycle
        int j = ((i + nums[i]) % n + n) % n;
        return j == i ? -1 : j; // single-element self-loop is invalid
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#485 Max Consecutive Ones

Description: Find the maximum number of consecutive 1s in a binary array.

Key: running counter, reset on 0, track the max.

Intuition: Grow a streak while seeing 1s and reset at each 0.

Variables: `best` = longest run of 1s · `cur` = current run length · `x` = scanned value **Pseudocode:**

```

best and current run zero
for each value x:
    if it's a 1:
        grow the run
        update the best
    else:
        reset the run

return the best

```

```

class Solution {
    public int findMaxConsecutiveOnes(int[] nums) {
        int best = 0, cur = 0;
        for (int x : nums) {
            if (x == 1) {
                cur++;
                best = Math.max(best, cur);
            } else {
                cur = 0;
            }
        }
        return best;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#493 Reverse Pairs

Description: Count pairs (i, j) with $i < j$ and $nums[i] > 2 * nums[j]$.

Key: merge sort; before merging each half, count reverse pairs across the sorted halves.

Intuition: Sorted halves let you count $nums[i] > 2 * nums[j]$ pairs with a linear two-pointer sweep per merge.

Variables: lo / hi = merge-sort range · mid = split · i / p = left/right merge pointers · j = counting pointer in the right half · $count$ = reverse pairs **Pseudocode:**

```

count over the whole array
merge sort:
    base case

    midpoint
    count within both halves
    counting pointer starts at the right half
    for each left element i:
        advance j while left > 2*right

        add how many right elements qualified
    merge the halves:

    while both have elements:

        take smaller left

        take smaller right

    drain left

    drain right

    copy back
    return the count

```

```

class Solution {
    public int reversePairs(int[] nums) {
        return mergeSort(nums, 0, nums.length - 1);
    }
    private int mergeSort(int[] nums, int lo, int hi) {
        if (lo >= hi) {
            return 0;
        }
        int mid = (lo + hi) >> 1;
        int count = mergeSort(nums, lo, mid) + mergeSort(nums, mid + 1, hi);
        int j = mid + 1;
        for (int i = lo; i <= mid; i++) {
            while (j <= hi && (long) nums[i] > 2L * nums[j]) {
                j++;
            }
            count += j - (mid + 1);
        }
        int[] merged = new int[hi - lo + 1];
        int i = lo, p = mid + 1, k = 0;
        while (i <= mid && p <= hi) {
            if (nums[i] <= nums[p]) {
                merged[k++] = nums[i++];
            } else {
                merged[k++] = nums[p++];
            }
        }
        while (i <= mid) {
            merged[k++] = nums[i++];
        }
        while (p <= hi) {
            merged[k++] = nums[p++];
        }
        System.arraycopy(merged, 0, nums, lo, merged.length);
        return count;
    }
}

```

Time $O(n \log n)$ | **Space** $O(n)$

#498 Diagonal Traverse

Description: Traverse an $m \times n$ matrix diagonally, alternating direction each diagonal.

Key: group by $r+c$; even diagonals go up-right, odd go down-left.

Intuition: Cells on the same diagonal share $r+c$; flip the read direction on alternating diagonals to zig-zag.

Variables: m / n = dims · idx = output index · r / c = current cell · up = direction (true = up-right) **Pseudocode:**

```
read dims
allocate the flat output
start at top-left, output index zero
begin going up-right
while not all cells emitted:
    emit the current cell
    if heading up-right:
        if at the right edge: drop down, flip direction

        else if at the top edge: step right, flip direction

        else: move up-right

    else (going down-left):
        if at the bottom edge: step right, flip direction

        else if at the left edge: step down, flip direction

        else: move down-left

return the result
```

```

class Solution {
    public int[] findDiagonalOrder(int[][] mat) {
        int m = mat.length, n = mat[0].length;
        int[] result = new int[m * n];
        int idx = 0, r = 0, c = 0;
        boolean up = true;
        while (idx < m * n) {
            result[idx++] = mat[r][c];
            if (up) {
                if (c == n - 1) {
                    r++;
                    up = false;
                } else if (r == 0) {
                    c++;
                    up = false;
                } else {
                    r--;
                    c++;
                }
            } else {
                if (r == m - 1) {
                    c++;
                    up = true;
                } else if (c == 0) {
                    r++;
                    up = true;
                } else {
                    r++;
                    c--;
                }
            }
        }
        return result;
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(1)$ excluding output

#517 Super Washing Machines

Description: Minimize the max number of moves to equalize dresses across machines (one dress to a neighbor per move).

Key: if total not divisible by n , impossible; answer is max of $|\text{prefixImbalance}|$ and any single overflow $\text{load} - \text{avg}$.

Intuition: A bottleneck is either a machine with too much surplus or a cut that must pass a large net flow.

Variables: total = sum of dresses · n = machines · avg = target each · running = net flow across the current cut · diff = surplus of one machine · result = answer **Pseudocode:**

sum all dresses

let n be the machine count

if not evenly divisible, impossible

target per machine

answer and running flow zero

for each machine:

 its surplus over average

 accumulate net flow across this cut

 bottleneck is the max of cut flow and single overflow

return the answer

```
class Solution {
    public int findMinMoves(int[] machines) {
        int total = 0;
        for (int m : machines) {
            total += m;
        }
        int n = machines.length;
        if (total % n != 0) {
            return -1;
        }
        int avg = total / n;
        int result = 0, running = 0;
        for (int m : machines) {
            int diff = m - avg;
            running += diff;
            result = Math.max(result, Math.max(Math.abs(running), diff));
        }
        return result;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#523 Continuous Subarray Sum

Description: Is there a subarray of length ≥ 2 whose sum is a multiple of k ?

Key: store earliest index for each prefix-sum mod k ; equal remainders far apart mean a divisible subarray.

Intuition: Two prefix sums with the same remainder mod k bound a subarray sum divisible by k .

Variables: `firstIndex` = map remainder mod k to earliest index · `sum` = running prefix · `rem` = normalized remainder

Pseudocode:

```
map remainder to earliest index
seed remainder 0 at index -1
running sum zero
for each index i:
    add the element
    compute the normalized remainder
    if this remainder was seen:
        if at least 2 apart, a valid subarray exists

    else record this remainder's index

no valid subarray
```

```
class Solution {
    public boolean checkSubarraySum(int[] nums, int k) {
        Map<Integer, Integer> firstIndex = new HashMap<>();
        firstIndex.put(0, -1);
        int sum = 0;
        for (int i = 0; i < nums.length; i++) {
            sum += nums[i];
            int rem = ((sum % k) + k) % k;
            if (firstIndex.containsKey(rem)) {
                if (i - firstIndex.get(rem) >= 2) {
                    return true;
                }
            } else {
                firstIndex.put(rem, i);
            }
        }
        return false;
    }
}
```

Time $O(n)$ | **Space** $O(\min(n, k))$

#525 Contiguous Array

Description: Find the longest subarray with equal numbers of 0s and 1s.

Key: treat 0 as -1; longest subarray with sum 0 via prefix-sum first-index map.

Intuition: Equal counts means a running balance returns to a value seen before.

Variables: `firstIndex` = map running balance to earliest index · `balance` = +1 per 1, -1 per 0 · `best` = longest length

Pseudocode:

```

map balance to earliest index
seed balance 0 at index -1
balance and best zero
for each index i:
    +1 for a 1, -1 for a 0
    if this balance was seen:
        candidate length from that earliest index
    else record this balance's index

return the best length

```

```

class Solution {
    public int findMaxLength(int[] nums) {
        Map<Integer, Integer> firstIndex = new HashMap<>();
        firstIndex.put(0, -1);
        int balance = 0, best = 0;
        for (int i = 0; i < nums.length; i++) {
            balance += (nums[i] == 1) ? 1 : -1;
            if (firstIndex.containsKey(balance)) {
                best = Math.max(best, i - firstIndex.get(balance));
            } else {
                firstIndex.put(balance, i);
            }
        }
        return best;
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#532 K-diff Pairs in an Array

Description: Count unique pairs (i, j) where $nums[i] - nums[j] == k$ ($k \geq 0$).

Key: frequency map; for $k=0$ count values with freq ≥ 2 , else count distinct values v with $v+k$ present.

Intuition: Counting distinct values avoids duplicate pairs; the zero-diff case needs a repeated value.

Variables: `count` = frequency map · `result` = unique-pair count · `e` = a (value, freq) entry **Pseudocode:**

```

count each value

result zero
for each distinct value entry:
    if k is zero:
        count it once if it appears at least twice

    else if value+k also exists, count the pair

return the count

```



```

class Solution {
    public int findPairs(int[] nums, int k) {
        Map<Integer, Integer> count = new HashMap<>();
        for (int x : nums) {
            count.merge(x, 1, Integer::sum);
        }
        int result = 0;
        for (Map.Entry<Integer, Integer> e : count.entrySet()) {
            if (k == 0) {
                if (e.getValue() >= 2) {
                    result++;
                }
            } else if (count.containsKey(e.getKey() + k)) {
                result++;
            }
        }
        return result;
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#560 Subarray Sum Equals K

Description: Count subarrays summing to exactly k.

Key: prefix-sum frequency map; for current sum `s` add count of `s - k` seen.

Intuition: Each earlier prefix equal to `s-k` marks the start of a qualifying subarray ending here.

Variables: `count` = map prefix sum to frequency · `sum` = running prefix · `result` = subarray count **Pseudocode:**

```

map prefix sum to frequency
seed sum 0 with count 1
running sum and result zero
for each value x:
    extend the running sum
    add how many earlier prefixes equal sum-k
    record this prefix
return the count

```

```

class Solution {
    public int subarraySum(int[] nums, int k) {
        Map<Integer, Integer> count = new HashMap<>();
        count.put(0, 1);
        int sum = 0, result = 0;
        for (int x : nums) {
            sum += x;
            result += count.getDefault(sum - k, 0);
            count.merge(sum, 1, Integer::sum);
        }
        return result;
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#566 Reshape the Matrix

Description: Reshape an $m \times n$ matrix into $r \times c$ keeping row-major order; return original if sizes mismatch.

Key: flatten index t ; source $(t/n, t\%n)$, target $(t/c, t\%c)$.

Intuition: Both shapes share a linear index, so map between them with division and modulo.

Variables: m/n = source dims · t = flat index; source $(t/n, t\%n)$, target $(t/c, t\%c)$ **Pseudocode:**

```
read source dims
if the element counts differ, return the original
```

```
allocate the reshaped grid
for each flat index t:
    map it from source layout to target layout
return the reshaped matrix
```

```
class Solution {
    public int[][] matrixReshape(int[][] mat, int r, int c) {
        int m = mat.length, n = mat[0].length;
        if (m * n != r * c) {
            return mat;
        }
        int[][] result = new int[r][c];
        for (int t = 0; t < m * n; t++) {
            result[t / c][t % c] = mat[t / n][t % n];
        }
        return result;
    }
}
```

Time $O(mn)$ | **Space** $O(rc)$ excluding output

#581 Shortest Unsorted Continuous Subarray

Description: Find the shortest subarray that, if sorted, makes the whole array sorted.

Key: scan from right tracking min to find left bound; scan from left tracking max to find right bound.

Intuition: The window starts where an element exceeds some later minimum and ends where an element falls below some earlier maximum.

Variables: n = length · $left / right$ = bounds of the unsorted window (default empty) · max / min = running extremes

Pseudocode:

```
let n be the length
default to an empty window
seed max from the left, min from the right
left-to-right, find the right bound:
    track the running max
    any element below it extends the right bound
right-to-left, find the left bound:
    track the running min
    any element above it extends the left bound
return the window length
```

```

class Solution {
    public int findUnsortedSubarray(int[] nums) {
        int n = nums.length;
        int left = -1, right = -2;    // default span length 0 when already sorted
        int max = nums[0], min = nums[n - 1];
        for (int i = 1; i < n; i++) {    // right bound = last element smaller than some earlier max
            max = Math.max(max, nums[i]);
            if (nums[i] < max) right = i;
        }
        for (int i = n - 2; i >= 0; i--) {    // left bound = first element larger than some later min
            min = Math.min(min, nums[i]);
            if (nums[i] > min) left = i;
        }
        return right - left + 1;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#594 Longest Harmonious Subsequence

Description: Find the longest subsequence where max and min differ by exactly 1.

Key: frequency map; for each value v , if $v+1$ exists, candidate length is `count[v] + count[v+1]`.

Intuition: A harmonious subsequence uses exactly two adjacent values, so combine each pair's counts.

Variables: `count` = frequency map · `best` = longest harmonious length · `e` = a (value, freq) entry **Pseudocode:**

count each value

best zero

for each value entry:

 if the value plus one exists:

 candidate is the sum of both counts

return the best

```

class Solution {
    public int findLHS(int[] nums) {
        Map<Integer, Integer> count = new HashMap<>();
        for (int x : nums) {
            count.merge(x, 1, Integer::sum);
        }
        int best = 0;
        for (Map.Entry<Integer, Integer> e : count.entrySet()) {
            if (count.containsKey(e.getKey() + 1)) {
                best = Math.max(best, e.getValue() + count.get(e.getKey() + 1));
            }
        }
        return best;
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#599 Minimum Index Sum of Two Lists

Description: Find common strings between two lists with minimum index sum.

Key: map first list value to index; scan second tracking the minimum index sum.

Intuition: Index sum is minimized greedily as you scan; ties are collected together.

Variables: `index` = map list1 string to its index · `result` = best matches · `best` = smallest index sum · `j` = list2 index

Pseudocode:

```
map each list1 string to its index

prepare the result list
best index sum is infinity
for each list2 index j:
    if this string is in list1:
        its index sum
        if strictly smaller, reset the result to just this

    else if tied, add it too

return the best matches
```

```
class Solution {
    public String[] findRestaurant(String[] list1, String[] list2) {
        Map<String, Integer> index = new HashMap<>();
        for (int i = 0; i < list1.length; i++) {
            index.put(list1[i], i);
        }
        List<String> result = new ArrayList<>();
        int best = Integer.MAX_VALUE;
        for (int j = 0; j < list2.length; j++) {
            if (index.containsKey(list2[j])) {
                int sum = j + index.get(list2[j]);
                if (sum < best) {
                    best = sum;
                    result.clear();
                    result.add(list2[j]);
                } else if (sum == best) {
                    result.add(list2[j]);
                }
            }
        }
        return result.toArray(new String[0]);
    }
}
```

Time $O(n + m)$ | **Space** $O(n)$

#611 Valid Triangle Number

Description: Count triplets from an array of side lengths that form a valid triangle.

Key: sort; fix the largest side k , opposite pointers on $[0, k-1]$; if $\text{nums}[i] + \text{nums}[j] > \text{nums}[k]$, all $i \dots j$ qualify.

Intuition: Only the two smaller sides need to beat the largest, so fix the largest and bulk-count valid pairs.

Variables: k = fixed largest side index · i / j = left/right over $[0, k-1]$ · count = valid triangles **Pseudocode:**

```
sort the sides
count zero
for each largest side k from the top:
    set inner pointers at the start and just below k
    while they haven't crossed:
        if the two smaller sides beat the largest:
            every i between also works, add j-i
            shrink the larger of the two
        else:
            grow the smaller of the two
return the count
```

```
class Solution {
    public int triangleNumber(int[] nums) {
        Arrays.sort(nums);
        int count = 0;
        for (int k = nums.length - 1; k >= 2; k--) {
            int i = 0, j = k - 1;
            while (i < j) {
                if (nums[i] + nums[j] > nums[k]) {
                    count += j - i;    // ← VARIATION: bulk-count valid i for this j
                    j--;
                } else {
                    i++;
                }
            }
        }
        return count;
    }
}
```

Time $O(n^2)$ | **Space** $O(1)$

#661 Image Smoother

Description: Replace each cell with the floor of the average of itself and its up-to-8 neighbors.

Key: for each cell sum valid neighbors (including self) and divide by the count.

Intuition: Average over the in-bounds 3×3 window centered on each cell.

Variables: m / n = dims · dr / dc = 8 neighbor offsets plus self · sum / cnt = neighborhood total and member count

Pseudocode:

```

read dims
allocate the result
neighbor row offsets including self
neighbor col offsets including self
for each cell (r,c):

    sum and count zero
    for each of the 9 offsets:

        if the neighbor is in bounds:
            add its value
            bump the count

    store the floored average

return the smoothed grid

```

```

class Solution {
public int[][] imageSmoother(int[][] img) {
    int m = img.length, n = img[0].length;
    int[][] result = new int[m][n];
    int[] dr = {1, -1, 0, 0, 1, 1, -1, -1, 0};
    int[] dc = {0, 0, 1, -1, 1, -1, 1, -1, 0};
    for (int r = 0; r < m; r++) {
        for (int c = 0; c < n; c++) {
            int sum = 0, cnt = 0;
            for (int dir = 0; dir < 9; dir++) {
                int nr = r + dr[dir], nc = c + dc[dir];
                if (nr >= 0 && nr < m && nc >= 0 && nc < n) {
                    sum += img[nr][nc];
                    cnt++;
                }
            }
            result[r][c] = sum / cnt;
        }
    }
    return result;
}
}

```

Time $O(mn)$ | **Space** $O(mn)$ excluding output

#723 Candy Crush

Description: Repeatedly crush horizontal/vertical runs of 3+ same-positive candies and apply gravity, until stable.

Key: mark crushable cells (negate), zero them, drop columns; repeat until no change.

Intuition: Mark all matches in one pass before clearing so simultaneous crushes are handled, then let candies fall.

Variables: m / n = dims · `changed` = whether any crush happened this round · r / c = scan cell · v = candidate value · `write` = gravity write row **Pseudocode:**

```
read dims
loop until a stable round

  reset the changed flag
  scan horizontal runs of 3:

    if three equal positive candies in a row:
      mark all three negative

  scan vertical runs of 3:

    if three equal positive candies in a column:
      mark all three negative

  if anything was marked, apply gravity per column:

    write row starts at the bottom
    for each row from the bottom up:
      if it's a surviving candy, drop it to the write row

    fill the rest of the column with zeros

return the stable board
```

```

class Solution {
    public int[][] candyCrush(int[][] board) {
        int m = board.length, n = board[0].length;
        boolean changed = true;
        while (changed) {
            changed = false;
            for (int r = 0; r < m; r++) {
                for (int c = 0; c + 2 < n; c++) {
                    int v = Math.abs(board[r][c]);
                    if (v != 0 && v == Math.abs(board[r][c + 1]) && v == Math.abs(board[r][c + 2])) {
                        board[r][c] = board[r][c + 1] = board[r][c + 2] = -v;
                        changed = true;
                    }
                }
            }
            for (int r = 0; r + 2 < m; r++) {
                for (int c = 0; c < n; c++) {
                    int v = Math.abs(board[r][c]);
                    if (v != 0 && v == Math.abs(board[r + 1][c]) && v == Math.abs(board[r + 2][c])) {
                        board[r][c] = board[r + 1][c] = board[r + 2][c] = -v;
                        changed = true;
                    }
                }
            }
            if (changed) {
                for (int c = 0; c < n; c++) {
                    int write = m - 1;
                    for (int r = m - 1; r >= 0; r--) {
                        if (board[r][c] > 0) {
                            board[write--][c] = board[r][c];
                        }
                    }
                    while (write >= 0) {
                        board[write--][c] = 0;
                    }
                }
            }
        }
        return board;
    }
}

```

Time $O((m*n)^2)$ worst case | **Space** $O(1)$

#724 Find Pivot Index

Description: Find the leftmost index where the sum to its left equals the sum to its right.

Key: total sum minus left and current gives right sum; check `left == total - left - nums[i]`.

Intuition: Track a running left sum and derive the right sum from the precomputed total.

Variables: `total` = full sum · `left` = running left-side sum; right side = `total - left - nums[i]` **Pseudocode:**

sum the whole array

left sum zero

for each index i:

if left equals total minus left minus current, it's the pivot

add the current element to the left sum

no pivot found

```
class Solution {
    public int pivotIndex(int[] nums) {
        int total = 0;
        for (int x : nums) {
            total += x;
        }
        int left = 0;
        for (int i = 0; i < nums.length; i++) {
            if (left == total - left - nums[i]) {
                return i;
            }
            left += nums[i];
        }
        return -1;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#766 Toeplitz Matrix

Description: Check if every top-left to bottom-right diagonal has a single value.

Key: each cell (except first row/col) must equal its up-left neighbor.

Intuition: Comparing each cell to its diagonal predecessor verifies all diagonals locally.

Variables: r / c = cell being compared to its up-left neighbor $(r-1, c-1)$ **Pseudocode:**

```
for each row from the second:
    for each col from the second:
        if a cell differs from its up-left neighbor:
            not Toeplitz
```

all diagonals consistent

```

class Solution {
    public boolean isToeplitzMatrix(int[][] matrix) {
        for (int r = 1; r < matrix.length; r++) {
            for (int c = 1; c < matrix[0].length; c++) {
                if (matrix[r][c] != matrix[r - 1][c - 1]) {
                    return false;
                }
            }
        }
        return true;
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(1)$

#769 Max Chunks To Make Sorted

Description: Array is a permutation of $[0, n-1]$. Max number of chunks that can be sorted independently to sort the whole.

Key: a chunk boundary occurs wherever the running max equals the current index.

Intuition: When the largest value seen so far equals the index, everything up to here is a self-contained block.

Variables: `max` = running max value seen · `chunks` = chunk count; boundary when `max == i` **Pseudocode:**

```

running max and chunk count zero
for each index i:
    update the running max
    if the max equals the index, close a chunk

return the chunk count

```

```

class Solution {
    public int maxChunksToSorted(int[] arr) {
        int max = 0, chunks = 0;
        for (int i = 0; i < arr.length; i++) {
            max = Math.max(max, arr[i]);
            if (max == i) {
                chunks++;
            }
        }
        return chunks;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#795 Number of Subarrays with Bounded Maximum

Description: Count subarrays whose maximum lies in $[left, right]$.

Key: count subarrays with $\max \leq right$ minus those with $\max \leq left-1$.

Intuition: "Max in range" equals the difference of two "max at most X" counts, each computed in one pass.

Variables: `bound` = upper limit for the helper · `count` = subarray count · `run` = length of the current valid run

Pseudocode:

```

answer = (max <= right) minus (max <= left-1)
helper counting subarrays with max at most bound:
    count and run zero
    for each value x:
        if within bound:
            extend the run
        else:
            reset the run

    every run of length r adds r new subarrays ending here

return the count

```

```

class Solution {
    public int numSubarrayBoundedMax(int[] nums, int left, int right) {
        return countMaxAtMost(nums, right) - countMaxAtMost(nums, left - 1);
    }
    private int countMaxAtMost(int[] nums, int bound) {
        int count = 0, run = 0;
        for (int x : nums) {
            if (x <= bound) {
                run++;
            } else {
                run = 0;
            }
            count += run;
        }
        return count;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#807 Max Increase to Keep City Skyline

Description: Increase building heights as much as possible without changing skylines viewed from any of 4 sides.

Key: each cell can rise to $\min(\text{rowMax}, \text{colMax})$; sum the increases.

Intuition: A building is capped by the smaller of its row and column maxima to preserve both silhouettes.

Variables: n = grid size · $\text{rowMax} / \text{colMax}$ = per-row and per-col maxima · total = sum of allowed increases

Pseudocode:

```

let n be the size
per-row maxima
per-col maxima
compute both maxima in one pass:

    update this row's max
    update this col's max

total zero
for each cell, raise it to min(rowMax, colMax):

    add the gained height

return the total

```

```

class Solution {
    public int maxIncreaseKeepingSkyline(int[][] grid) {
        int n = grid.length;
        int[] rowMax = new int[n];
        int[] colMax = new int[n];
        for (int r = 0; r < n; r++) {
            for (int c = 0; c < n; c++) {
                rowMax[r] = Math.max(rowMax[r], grid[r][c]);
                colMax[c] = Math.max(colMax[c], grid[r][c]);
            }
        }
        int total = 0;
        for (int r = 0; r < n; r++) {
            for (int c = 0; c < n; c++) {
                total += Math.min(rowMax[r], colMax[c]) - grid[r][c];
            }
        }
        return total;
    }
}

```

Time $O(n^2)$ | **Space** $O(n)$

#825 Friends Of Appropriate Ages

Description: Count friend requests; x friends y unless $\text{age}[y] \leq 0.5 \cdot \text{age}[x] + 7$, $\text{age}[y] > \text{age}[x]$, or $\text{age}[y] > 100$ && $\text{age}[x] < 100$.

Key: bucket by age (1..120); for each pair of ages count valid requests.

Intuition: Ages are bounded, so counting per age-bucket pair collapses the problem to constant-size work.

Variables: `count` = per-age population (1..120) · `x / y` = requester/target age buckets · `result` = total requests

Pseudocode:

bucket population by age

result zero

for each requester age x:

for each target age y:

skip if the request rules forbid it

add all x-by-y pairs

if same age, remove self-requests

return the request count

```
class Solution {
    public int numFriendRequests(int[] ages) {
        int[] count = new int[121];
        for (int a : ages) {
            count[a]++;
        }
        int result = 0;
        for (int x = 1; x <= 120; x++) {
            for (int y = 1; y <= 120; y++) {
                if (y <= 0.5 * x + 7 || y > x || (y > 100 && x < 100)) {
                    continue;
                }
                result += count[x] * count[y];
                if (x == y) {
                    result -= count[x];
                }
            }
        }
        return result;
    }
}
```

Time $O(n + 120^2)$ | **Space** $O(1)$

#838 Push Dominoes

Description: Simulate falling dominoes given a string of 'L', 'R', '.'.

Key: compute net force at each cell — R pushes right (+, decaying), L pushes left (-, decaying); sign decides final state.

Intuition: Each cell's outcome is the balance of forces from the nearest pushers on either side.

Variables: n = length · force = net force per cell (+ right, - left) · f = decaying push strength **Pseudocode:**

```
let n be the length
net force per cell
push strength zero
left-to-right pass for rightward forces:
  read the cell
  R starts a full-strength rightward push

  L kills any rightward push

  a gap decays the push toward zero

  add the rightward force
reset strength
right-to-left pass for leftward forces:
  read the cell
  L starts a full-strength leftward push

  R kills any leftward push

  a gap decays it

  subtract the leftward force
build the result from each cell's net force:
  positive -> R, negative -> L, zero -> upright
return the string
```

```

class Solution {
    public String pushDominoes(String dominoes) {
        int n = dominoes.length();
        int[] force = new int[n];
        int f = 0;
        for (int i = 0; i < n; i++) {
            char ch = dominoes.charAt(i);
            if (ch == 'R') {
                f = n;
            } else if (ch == 'L') {
                f = 0;
            } else {
                f = Math.max(f - 1, 0);
            }
            force[i] += f;
        }
        f = 0;
        for (int i = n - 1; i >= 0; i--) {
            char ch = dominoes.charAt(i);
            if (ch == 'L') {
                f = n;
            } else if (ch == 'R') {
                f = 0;
            } else {
                f = Math.max(f - 1, 0);
            }
            force[i] -= f;
        }
        StringBuilder builder = new StringBuilder();
        for (int x : force) {
            builder.append(x > 0 ? 'R' : x < 0 ? 'L' : '.');
        }
        return builder.toString();
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#845 Longest Mountain in Array

Description: Find the length of the longest mountain (strictly up then strictly down).

Key: find each up-slope start, extend up then down; a valid mountain needs both slopes.

Intuition: Scan to a peak by rising, then descend; the span counts only when both directions occurred.

Variables: `n` = length · `best` = longest mountain · `i` = scan index · `start` = foot of the climb · `peak` = top of the climb
Pseudocode:

```

length, best, scan index zero
scan the array:
    remember the foot of a potential mountain
    climb while strictly rising
    if there was an up-slope:
        remember the peak
        descend while strictly falling
        if there was a down-slope too, it's a mountain; update best

    if nothing was consumed, force progress
return the best length

```

```

class Solution {
    public int longestMountain(int[] arr) {
        int n = arr.length, best = 0, i = 0;
        while (i < n) {
            int start = i;
            while (i + 1 < n && arr[i] < arr[i + 1]) i++;    // climb up
            if (i > start) {                                // had an up-slope: try the down-slope
                int peak = i;
                while (i + 1 < n && arr[i] > arr[i + 1]) i++;
                if (i > peak) best = Math.max(best, i - start + 1);    // both slopes present → mountain
            }
            if (i == start) i++;    // flat or descent from start: nothing consumed, force progress
        }
        return best;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#896 Monotonic Array

Description: Return true if the array is entirely non-increasing or entirely non-decreasing.

Key: track whether any increase and any decrease occurred; monotonic iff not both.

Intuition: A single pass noting direction changes detects non-monotonicity.

Variables: `inc / dec` = whether any increase / decrease occurred · `i` = scan index **Pseudocode:**

```

increase-seen and decrease-seen flags false
for each adjacent pair:
    if it rises, note an increase

    else if it falls, note a decrease

monotonic unless both directions appeared

```



```

class Solution {
    public boolean isMonotonic(int[] nums) {
        boolean inc = false, dec = false;
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] > nums[i - 1]) {
                inc = true;
            } else if (nums[i] < nums[i - 1]) {
                dec = true;
            }
        }
        return !(inc && dec);
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#912 Sort an Array

Description: Sort an integer array. Implement an $O(n \log n)$ sort.

Key: merge sort with a reusable buffer.

Intuition: Recursively split, sort halves, and merge — stable $O(n \log n)$ without library sort.

Variables: `lo / hi` = sort range · `mid` = split · `buf` = shared merge buffer · `i / j` = half pointers · `k` = buffer write index
Pseudocode:

```

sort over the whole array with a shared buffer

merge sort:
    base case single element

    midpoint
    sort the left half
    sort the right half
    set the two half pointers and the buffer index
    while both halves have elements:
        take the smaller front element:
            take from the left

            take from the right

    drain the left half

    drain the right half

    copy the merged run back

```

```

class Solution {
    public int[] sortArray(int[] nums) {
        mergeSort(nums, 0, nums.length - 1, new int[nums.length]);
        return nums;
    }
    private void mergeSort(int[] nums, int lo, int hi, int[] buf) {
        if (lo >= hi) {
            return;
        }
        int mid = (lo + hi) >> 1;
        mergeSort(nums, lo, mid, buf);
        mergeSort(nums, mid + 1, hi, buf);
        int i = lo, j = mid + 1, k = lo;
        while (i <= mid && j <= hi) {
            if (nums[i] <= nums[j]) {
                buf[k++] = nums[i++];
            } else {
                buf[k++] = nums[j++];
            }
        }
        while (i <= mid) {
            buf[k++] = nums[i++];
        }
        while (j <= hi) {
            buf[k++] = nums[j++];
        }
        System.arraycopy(buf, lo, nums, lo, hi - lo + 1);
    }
}

```

Time $O(n \log n)$ | **Space** $O(n)$

#974 Subarray Sums Divisible by K

Description: Count subarrays whose sum is divisible by k.

Key: prefix-sum remainder counts; subarrays between equal remainders are divisible.

Intuition: Two prefixes with the same remainder mod k bound a divisible subarray; count pairs per remainder.

Variables: `count` = remainder-frequency array · `sum` = running prefix · `rem` = normalized remainder · `result` = subarray count
Pseudocode:

```

remainder counts, seed remainder 0 with count 1

running sum and result zero
for each value x:
    extend the running sum
    compute the normalized remainder
    add how many earlier prefixes share this remainder
    record this remainder
return the count

```

```

class Solution {
    public int subarraysDivByK(int[] nums, int k) {
        int[] count = new int[k];
        count[0] = 1;
        int sum = 0, result = 0;
        for (int x : nums) {
            sum += x;
            int rem = ((sum % k) + k) % k;
            result += count[rem];
            count[rem]++;
        }
        return result;
    }
}

```

Time $O(n)$ | **Space** $O(k)$

#977 Squares of a Sorted Array

Description: Return the squares of a sorted array in sorted order.

Key: opposite pointers; the larger absolute value is at one of the two ends — write from the back.

Intuition: The biggest squares sit at the extremes, so fill the result from the largest slot inward.

Variables: n = length · $result$ = output · i / j = left/right pointers · k = write index filling from the back

Pseudocode:

```

let n be the length
allocate the output
pointers at both ends, write index at the back
while pointers haven't crossed:
    square both ends
    if the left square is larger:
        write it at the back
        advance left
    else:
        write the right square at the back
        retreat right
return the result

```

```

class Solution {
    public int[] sortedSquares(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        int i = 0, j = n - 1, k = n - 1;
        while (i <= j) {
            int sqI = nums[i] * nums[i], sqJ = nums[j] * nums[j];
            if (sqI > sqJ) {
                result[k--] = sqI;
                i++;
            } else {
                result[k--] = sqJ;
                j--;
            }
        }
        return result;
    }
}

```

Time $O(n)$ | **Space** $O(1)$ excluding output

#1013 Partition Array Into Three Parts With Equal Sum

Description: Can the array be split into three contiguous parts with equal sums?

Key: total must be divisible by 3; sweep accumulating, counting completed $\text{total}/3$ parts.

Intuition: Greedily close off a part each time the running sum hits one-third; succeed if at least three parts form.

Variables: `total` = full sum · `target` = $\text{total}/3$ · `running` = current part sum · `parts` = completed parts

Pseudocode:

```

sum the whole array

if not divisible by 3, impossible

target, running sum, parts count
for each index i:
    extend the running sum
    if it reaches one-third:
        close a part and reset the running sum

    if two parts are closed with elements left, success

otherwise fail

```

```

class Solution {
    public boolean canThreePartsEqualSum(int[] arr) {
        int total = 0;
        for (int x : arr) {
            total += x;
        }
        if (total % 3 != 0) {
            return false;
        }
        int target = total / 3, running = 0, parts = 0;
        for (int i = 0; i < arr.length; i++) {
            running += arr[i];
            if (running == target) {
                parts++;
                running = 0;
                if (parts == 2 && i < arr.length - 1) {
                    return true;
                }
            }
        }
        return false;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#1074 Number of Submatrices That Sum to Target

Description: Count submatrices summing to target.

Key: fix a pair of rows, collapse columns to a 1D array, then apply prefix-sum hashmap (#560) per column.

Intuition: Reduce 2D to 1D by fixing the top and bottom rows, then count target-sum subarrays.

Variables: m / n = dims · `prefix` = column-prefix sums (rows padded) · `top / bottom` = fixed row band · `count` = map collapsed-sum to frequency · `sum` = running 1D prefix · `result` = answer **Pseudocode:**

```

read dims
column-prefix sums, padded by a top row
build them:

    prefix[r+1][c] = prefix above plus this cell

result zero
for each top row:
    for each bottom row below it:
        fresh prefix-sum map seeded with 0->1

        running 1D sum zero
        sweep columns:
            add this column's band sum
            add earlier prefixes equal to sum-target
            record this prefix

return the count

```

```

class Solution {
    public int numSubmatrixSumTarget(int[][] matrix, int target) {
        int m = matrix.length, n = matrix[0].length;
        int[][] prefix = new int[m + 1][n];
        for (int r = 0; r < m; r++) {
            for (int c = 0; c < n; c++) {
                prefix[r + 1][c] = prefix[r][c] + matrix[r][c];
            }
        }
        int result = 0;
        for (int top = 0; top < m; top++) {
            for (int bottom = top + 1; bottom <= m; bottom++) {
                Map<Integer, Integer> count = new HashMap<>();
                count.put(0, 1);
                int sum = 0;
                for (int c = 0; c < n; c++) {
                    sum += prefix[bottom][c] - prefix[top][c];
                    result += count.containsKey(sum - target) ? count.get(sum - target) : 0;
                    count.merge(sum, 1, Integer::sum);
                }
            }
        }
        return result;
    }
}

```

Time $O(m^2 * n)$ | **Space** $O(n)$

#1109 Corporate Flight Bookings

Description: Given bookings `[first, last, seats]`, return total seats booked per flight.

Key: difference array — add seats at `first`, subtract at `last+1`, prefix-sum.

Intuition: Range increments become two endpoint marks resolved by one prefix pass.

Variables: `diff` = difference array (one longer) · `running` = prefix accumulator · `result` = per-flight totals

Pseudocode:

```

allocate a difference array
for each booking [first,last,seats]:
    add seats at first (0-based)
    subtract seats just past last
allocate the result
running total zero
prefix-sum into the result:
    add this slot's delta
    store the running total
return the result

```

```

class Solution {
    public int[] corpFlightBookings(int[][] bookings, int n) {
        int[] diff = new int[n + 1];
        for (int[] b : bookings) {
            diff[b[0] - 1] += b[2];
            diff[b[1]] -= b[2];
        }
        int[] result = new int[n];
        int running = 0;
        for (int i = 0; i < n; i++) {
            running += diff[i];
            result[i] = running;
        }
        return result;
    }
}

```

Time $O(n + \text{bookings})$ | **Space** $O(n)$

#1213 Intersection of Three Sorted Arrays

Description: Return the common elements of three sorted arrays.

Key: three pointers; advance the smallest; record when all three match.

Intuition: Like merging — advance whichever pointer lags so all three meet at shared values.

Variables: `i / j / k` = independent pointers into the three sorted arrays · `result` = common values **Pseudocode:**

```

prepare the result list
all three pointers at the start
while all three are in range:
    if all three values match:
        record it and advance all three

    else if arr1's value is the smallest, advance it

    else if arr2's value is the smallest, advance it

    else advance arr3

return the common values

```

```

class Solution {
    public List<Integer> arraysIntersection(int[] arr1, int[] arr2, int[] arr3) {
        List<Integer> result = new ArrayList<>();
        int i = 0, j = 0, k = 0;
        while (i < arr1.length && j < arr2.length && k < arr3.length) {
            if (arr1[i] == arr2[j] && arr2[j] == arr3[k]) {
                result.add(arr1[i]);
                i++;
                j++;
                k++;
            } else if (arr1[i] <= arr2[j] && arr1[i] <= arr3[k]) {
                i++;
            } else if (arr2[j] <= arr1[i] && arr2[j] <= arr3[k]) {
                j++;
            } else {
                k++;
            }
        }
        return result;
    }
}

```

Time O(n) | **Space** O(1) excluding output

#1275 Find Winner on a Tic Tac Toe Game

Description: Given alternating A/B moves, determine the winner, "Draw", or "Pending".

Key: tally row/col/diagonal sums with +1 for A and -1 for B; $|\text{sum}| == 3$ wins.

Intuition: Signed counts per line reveal a win the moment any line reaches +3 or -3.

Variables: `rows / cols` = signed per-line sums · `diag / anti` = main/anti diagonal sums · `t` = move index · `sign` = +1 for A, -1 for B **Pseudocode:**

```

row sums
column sums
diagonal sums zero
for each move t:
    +1 on even (A), -1 on odd (B)
    read its row and col
    add to that row
    add to that col
    if on the main diagonal, add to it

    if on the anti-diagonal, add to it

    if any line reaches +-3:
        the current player won

full board is a draw, otherwise pending

```



```

class Solution {
    public String tictactoe(int[][] moves) {
        int[] rows = new int[3];
        int[] cols = new int[3];
        int diag = 0, anti = 0;
        for (int t = 0; t < moves.length; t++) {
            int sign = (t % 2 == 0) ? 1 : -1;
            int r = moves[t][0], c = moves[t][1];
            rows[r] += sign;
            cols[c] += sign;
            if (r == c) {
                diag += sign;
            }
            if (r + c == 2) {
                anti += sign;
            }
            if (Math.abs(rows[r]) == 3 || Math.abs(cols[c]) == 3
                || Math.abs(diag) == 3 || Math.abs(anti) == 3) {
                return sign == 1 ? "A" : "B";
            }
        }
        return moves.length == 9 ? "Draw" : "Pending";
    }
}

```

Time O(1) | **Space** O(1)

#1351 Count Negative Numbers in a Sorted Matrix

Description: Count negatives in a matrix sorted descending by row and column.

Key: staircase walk from bottom-left; on a negative move up, else move right.

Intuition: From the bottom-left, sorted order lets each step rule out a whole row or column of negatives.

Variables: `m / n` = dims · `r / c` = staircase position starting bottom-left · `count` = negatives found **Pseudocode:**

```

read dims
start at bottom-left, count zero
while in the grid:
    if the cell is negative:
        everything to the right in this row is negative too
        move up a row
    else:
        move right

return the count

```

```

class Solution {
    public int countNegatives(int[][] grid) {
        int m = grid.length, n = grid[0].length;
        int r = m - 1, c = 0, count = 0;
        while (r >= 0 && c < n) {
            if (grid[r][c] < 0) {
                count += n - c;
                r--;
            } else {
                c++;
            }
        }
        return count;
    }
}

```

Time $O(m + n)$ | **Space** $O(1)$

#1424 Diagonal Traverse II

Description: Traverse a jagged 2D list diagonally from bottom-left to top-right.

Key: group cells by $r+c$; within a diagonal, larger row comes first (bottom-left start).

Intuition: Cells on a diagonal share $r+c$; emitting rows in descending order gives the bottom-left-up direction.

Variables: `diagonals` = map $r+c$ to its cell list · `total` = element count · `maxKey` = largest diagonal key · `idx` = output write index **Pseudocode:**

```

map diagonal key to its values
total and max key zero
scan every cell:

    diagonal key is r+c
    append the cell to that diagonal
    track the largest key
    bump the total

allocate the flat output
output index zero
for each diagonal key in order:
    fetch its list
    skip if empty

    emit it bottom-up (reversed) for the right direction

return the result

```

```

class Solution {
    public int[] findDiagonalOrder(List<List<Integer>> nums) {
        Map<Integer, List<Integer>> diagonals = new HashMap<>();
        int total = 0, maxKey = 0;
        for (int r = 0; r < nums.size(); r++) {
            for (int c = 0; c < nums.get(r).size(); c++) {
                int key = r + c;
                diagonals.computeIfAbsent(key, x -> new ArrayList<>()).add(nums.get(r).get(c));
                maxKey = Math.max(maxKey, key);
                total++;
            }
        }
        int[] result = new int[total];
        int idx = 0;
        for (int key = 0; key <= maxKey; key++) {
            List<Integer> diag = diagonals.get(key);
            if (diag == null) {
                continue;
            }
            for (int i = diag.size() - 1; i >= 0; i--) {
                result[idx++] = diag.get(i);
            }
        }
        return result;
    }
}

```

Time $O(\text{total})$ | **Space** $O(\text{total})$

#1460 Make Two Arrays Equal by Reversing Sub-arrays

Description: Can target become arr after reversing any sub-arrays any number of times?

Key: reversals allow arbitrary reordering, so the arrays just need identical multisets.

Intuition: Any permutation is reachable through reversals, so only element counts matter.

Variables: `count` = per-value frequency difference (1001 buckets) · `x` = scanned value **Pseudocode:**

```

frequency-difference buckets
add each target value

```

```

subtract each arr value

```

```

if any bucket is non-zero, the multisets differ

```

```

they match

```

```

class Solution {
    public boolean canBeEqual(int[] target, int[] arr) {
        int[] count = new int[1001];
        for (int x : target) {
            count[x]++;
        }
        for (int x : arr) {
            count[x]--;
        }
        for (int c : count) {
            if (c != 0) {
                return false;
            }
        }
        return true;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#1480 Running Sum of 1d Array

Description: Return the running (prefix) sum of the array.

Key: accumulate in-place, each element adds the previous.

Intuition: Each running sum is the prior running sum plus the current value.

Variables: `i` = scan index; each element absorbs the previous running sum **Pseudocode:**

```

from the second element on:
    add the previous running sum into this slot
return the array (now prefix sums)

```

```

class Solution {
    public int[] runningSum(int[] nums) {
        for (int i = 1; i < nums.length; i++) {
            nums[i] += nums[i - 1];
        }
        return nums;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#1498 Number of Subsequences That Satisfy the Given Sum Condition

Description: Count subsequences where $\min + \max \leq \text{target}$. Answer mod $1e9+7$.

Key: sort; opposite pointers — if $\text{nums}[i] + \text{nums}[j] \leq \text{target}$, all subsets of $(i, j]$ with i included count $\rightarrow 2^{(j-i)}$.

Intuition: After sorting, fixing the minimum lets every subset of the elements up to the max be chosen freely.

Variables: `MOD` = $1e9+7$ · `n` = length · `pow` = precomputed powers of two · `i / j` = left/right pointers · `result` = subsequence count **Pseudocode:**

modulus and length

sort so min/max are the pointer ends

precompute powers of two:

pointers at both ends, result zero

while they haven't crossed:

if min plus max fits the target:

every subset between adds $2^{(j-i)}$

advance the left (min)

else:

retreat the right (max)

return the count

```
class Solution {
    public int numSubseq(int[] nums, int target) {
        int MOD = 1_000_000_007, n = nums.length;
        Arrays.sort(nums);
        long[] pow = new long[n];
        pow[0] = 1;
        for (int i = 1; i < n; i++) {
            pow[i] = pow[i - 1] * 2 % MOD;
        }
        int i = 0, j = n - 1;
        long result = 0;
        while (i <= j) {
            if (nums[i] + nums[j] <= target) {
                result = (result + pow[j - i]) % MOD;
                i++;
            } else {
                j--;
            }
        }
        return (int) result;
    }
}
```

Time $O(n \log n)$ | **Space** $O(n)$

#1748 Sum of Unique Elements

Description: Sum all elements that appear exactly once.

Key: frequency count; add values with count == 1.

Intuition: Count occurrences, then sum only the singletons.

Variables: count = per-value frequency (1..100) · x = scanned value · v = value being summed · sum = total of singletons
Pseudocode:

frequency buckets
count each value

sum zero
for each possible value v:
 if it occurred exactly once, add it

return the sum

```
class Solution {  
    public int sumOfUnique(int[] nums) {  
        int[] count = new int[101];  
        for (int x : nums) {  
            count[x]++;  
        }  
        int sum = 0;  
        for (int v = 1; v <= 100; v++) {  
            if (count[v] == 1) {  
                sum += v;  
            }  
        }  
        return sum;  
    }  
}
```

Time $O(n)$ | **Space** $O(1)$

#1762 Buildings With an Ocean View

Description: Return indices of buildings with nothing taller to their right (ocean to the right).

Key: scan right-to-left tracking max height; a building qualifies if taller than all seen so far.

Intuition: Sweeping from the ocean side, only record buildings that exceed every taller one already passed.

Variables: `result` = qualifying indices (collected back-to-front) · `max` = tallest building seen from the right · `arr` = final output
Pseudocode:

```
prepare the result list  
max height seen zero  
scan from the right (ocean) side:  
    if this building beats everything to its right:  
        record it  
        raise the max
```

copy to an array in increasing-index order

return it

```

class Solution {
    public int[] findBuildings(int[] heights) {
        List<Integer> result = new ArrayList<>();
        int max = 0;
        for (int i = heights.length - 1; i >= 0; i--) {
            if (heights[i] > max) {
                result.add(i);
                max = heights[i];
            }
        }
        int[] arr = new int[result.size()];
        for (int i = 0; i < arr.length; i++) {
            arr[i] = result.get(arr.length - 1 - i);
        }
        return arr;
    }
}

```

Time $O(n)$ | **Space** $O(1)$ excluding output

#1868 Product of Two Run-Length Encoded Arrays

Description: Multiply two run-length-encoded arrays element-wise; return the RLE product.

Key: two pointers over the runs; multiply values, take the min remaining length, decrement, merge equal adjacent products.

Intuition: March through both run lists together, consuming the shorter overlap and coalescing equal results.

Variables: `result` = output runs · `i / j` = run pointers into the two encodings · `product` = value of the overlap · `len` = overlap length · `last` = last result run **Pseudocode:**

```

prepare the result list
both run pointers at the start
while both have runs left:
    product of the two current run values
    overlap length = the shorter remaining run
    if it matches the previous result run's value:
        extend that run

    else:
        start a new run

    consume the overlap from both runs

    if the first run is used up, advance it

    if the second run is used up, advance it

return the merged runs

```

```

class Solution {
    public List<List<Integer>> findRLEArray(int[][] encoded1, int[][] encoded2) {
        List<List<Integer>> result = new ArrayList<>();
        int i = 0, j = 0;
        while (i < encoded1.length && j < encoded2.length) {
            int product = encoded1[i][0] * encoded2[j][0];
            int len = Math.min(encoded1[i][1], encoded2[j][1]);
            if (!result.isEmpty() && result.get(result.size() - 1).get(0) == product) {
                List<Integer> last = result.get(result.size() - 1);
                last.set(1, last.get(1) + len);
            } else {
                result.add(new ArrayList<>(Arrays.asList(product, len)));
            }
            encoded1[i][1] -= len;
            encoded2[j][1] -= len;
            if (encoded1[i][1] == 0) {
                i++;
            }
            if (encoded2[j][1] == 0) {
                j++;
            }
        }
        return result;
    }
}

```

Time $O(n + m)$ | **Space** $O(1)$ excluding output

#1877 Minimize Maximum Pair Sum in Array

Description: Pair up elements to minimize the maximum pair sum.

Key: sort, pair smallest with largest via opposite pointers; track the max pair sum.

Intuition: Pairing extremes balances the sums, keeping the largest pair as small as possible.

Variables: i / j = left/right pointers after sorting · max = largest pair sum **Pseudocode:**

```

sort the array
pointers at both ends, max zero
while they haven't crossed:
    pair the extremes and track the largest sum
    advance left
    retreat right
return the max pair sum

```

```

class Solution {
    public int minPairSum(int[] nums) {
        Arrays.sort(nums);
        int i = 0, j = nums.length - 1, max = 0;
        while (i < j) {
            max = Math.max(max, nums[i] + nums[j]);
            i++;
            j--;
        }
        return max;
    }
}

```

Time $O(n \log n)$ | **Space** $O(1)$

#1893 Check if All the Integers in a Range Are Covered

Description: Are all integers in $[left, right]$ covered by at least one given interval?

Key: difference array over the small value domain (1..50); prefix-sum to test coverage.

Intuition: Mark interval starts and ends, then a prefix sweep shows which points are covered.

Variables: `diff` = difference array over values 1..50 · `running` = coverage at the current point · `i` = value being checked
Pseudocode:

```
difference array over the value domain
for each interval:
    +1 at its start
    -1 just past its end
running coverage zero
sweep values 1..50:
    accumulate coverage
    if a queried value has zero coverage, fail

all queried values covered
```

```
class Solution {
    public boolean isCovered(int[][] ranges, int left, int right) {
        int[] diff = new int[52];
        for (int[] r : ranges) {
            diff[r[0]]++;
            diff[r[1] + 1]--;
        }
        int running = 0;
        for (int i = 1; i <= 50; i++) {
            running += diff[i];
            if (i >= left && i <= right && running <= 0) {
                return false;
            }
        }
        return true;
    }
}
```

Time $O(n + 50)$ | **Space** $O(1)$

#1894 Find the Student that Will Replace the Chalk

Description: Students consume chalk in order cyclically; find who runs out.

Key: reduce k modulo the total sum, then sweep once.

Intuition: Whole cycles repeat, so only the remainder after a full round determines the failing student.

Variables: `total` = chalk per full cycle · `k` = remaining chalk after whole cycles · `i` = student index
Pseudocode:

sum chalk over one full round

reduce k by whole cycles

walk students in order:

if this student can't be served, they run out

subtract their consumption

wrap to the first student

```
class Solution {
    public int chalkReplacer(int[] chalk, int k) {
        long total = 0;
        for (int c : chalk) {
            total += c;
        }
        k = (int) (k % total);
        for (int i = 0; i < chalk.length; i++) {
            if (k < chalk[i]) {
                return i;
            }
            k -= chalk[i];
        }
        return 0;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#1920 Build Array from Permutation

Description: Return `result[i] = nums[nums[i]]`.

Key: straightforward indexed build ($O(n)$ extra space) or encode two values per slot for $O(1)$ space.

Intuition: Each output is a double lookup into the permutation.

Variables: `result` = output · `i` = scan index; each slot is a double lookup **Pseudocode:**

```
allocate the result
for each index i:
    result[i] = the value at nums[nums[i]]

return the result
```

```
class Solution {
    public int[] buildArray(int[] nums) {
        int[] result = new int[nums.length];
        for (int i = 0; i < nums.length; i++) {
            result[i] = nums[nums[i]];
        }
        return result;
    }
}
```

Time $O(n)$ | **Space** $O(n)$ excluding output

#1929 Concatenation of Array

Description: Return `nums` concatenated with itself.

Key: allocate $2n$ and copy `nums` into both halves.

Intuition: Copy each element into position `i` and `i+n`.

Variables: `n` = length · `result` = doubled output · `i` = scan index; copy into slots `i` and `i+n` **Pseudocode:**

```
let n be the length
allocate an array of double length
for each index i:
    copy into the first half
    copy into the second half

return the concatenation
```

```
class Solution {
    public int[] getConcatenation(int[] nums) {
        int n = nums.length;
        int[] result = new int[2 * n];
        for (int i = 0; i < n; i++) {
            result[i] = nums[i];
            result[i + n] = nums[i];
        }
        return result;
    }
}
```

Time $O(n)$ | **Space** $O(1)$ excluding output